ECE 491 Image Processing and Computer Vision (Fall 2026)



Lecture 22 ConvNets, Transformers and Transfer Learning

Instructor: Tianfu (Matt) Wu, Associate Professor in ECE

PI of the iVMCL lab (interpretable Visual Modeling, Computing and Learning)

Website: https://research.ece.ncsu.edu/ivmcl/

Code & Models: https://github.com/iVMCL/

NC STATE UNIVERSITY Electrical and Computer Engineering

Outline

- **Topics:** from MLP to convolution; convolution layers and feature maps; the ConvNet roadmap (LeNet to ResNet); attention and Vision Transformers; transfer learning.
- **Key concept:** build the right structure into the network (locality, weight sharing), or learn it from data at scale, and then reuse what was learned.
- **Notes:** notebook 26 (marimo): shift test, pretrained ResNet-18, attention from scratch, transfer learning on CIFAR-10.

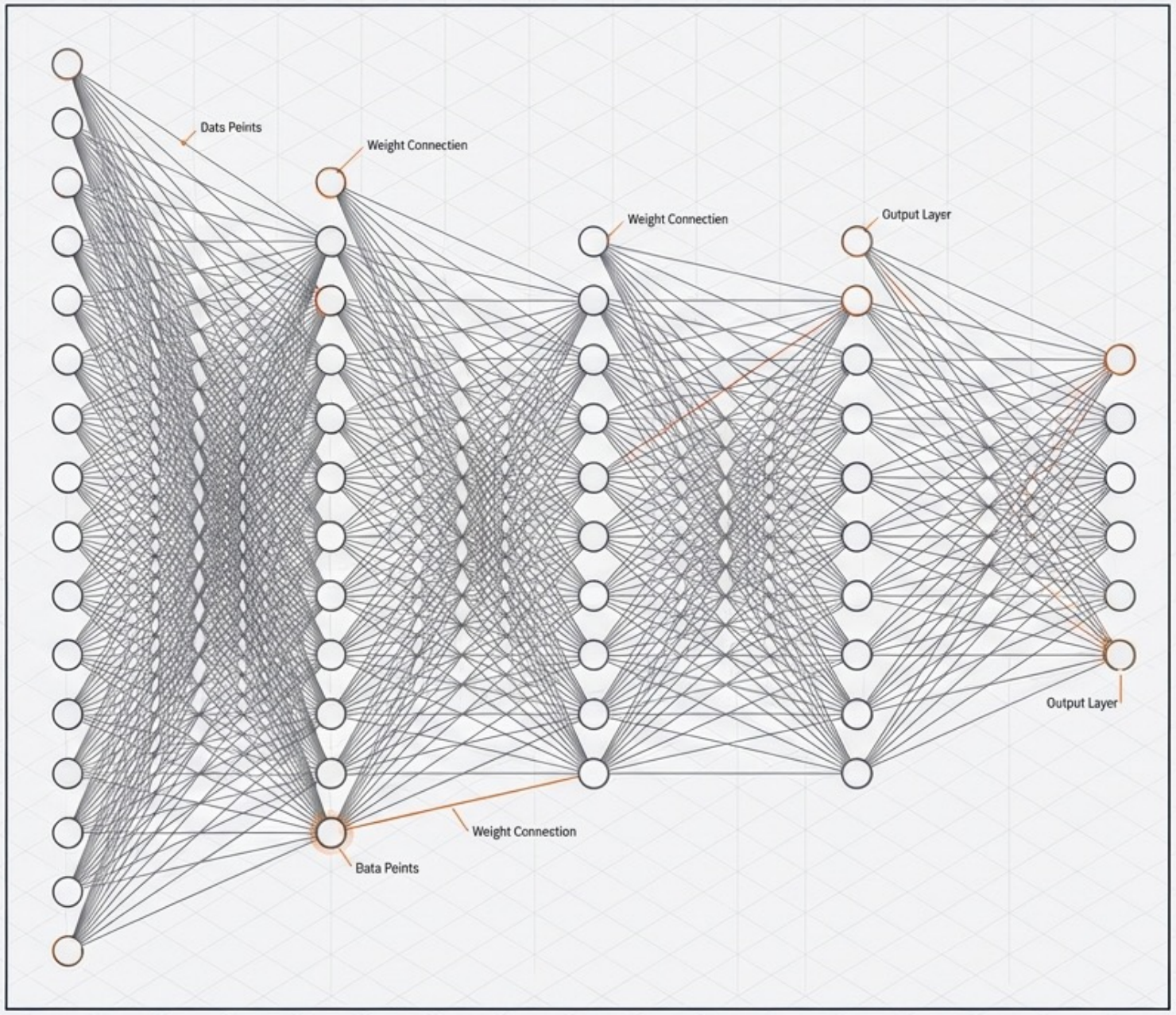
The Curse of Dimensionality

Scenario: Standard Image Input (3 x 224 x 224)

Target: Hidden Layer (64 x 56 x 56)

Input Size: 150,528 values
Hidden Layer: 200,704 values
Total Weights: ~950 Million

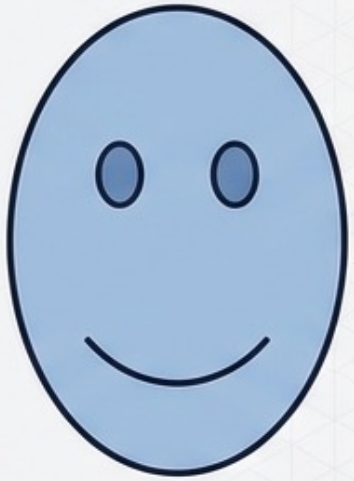
Status: **Computationally Prohibitive**



NotebookLM

Flattening Destroys Spatial Context

Original Image
(2D Structure)

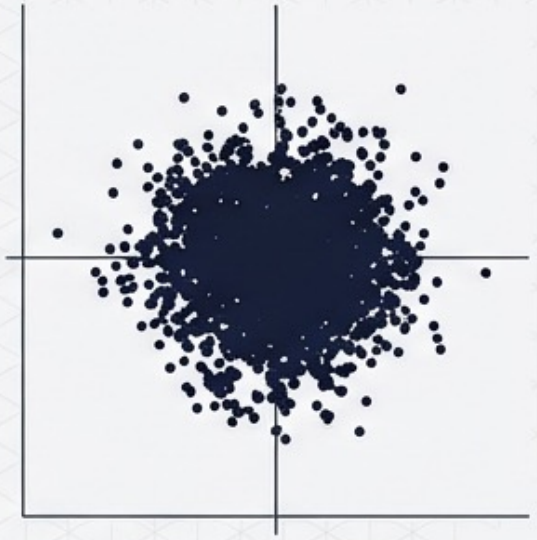


FLATTENING / UNROLLING

1D Vector
(Loss of Spatial Info)



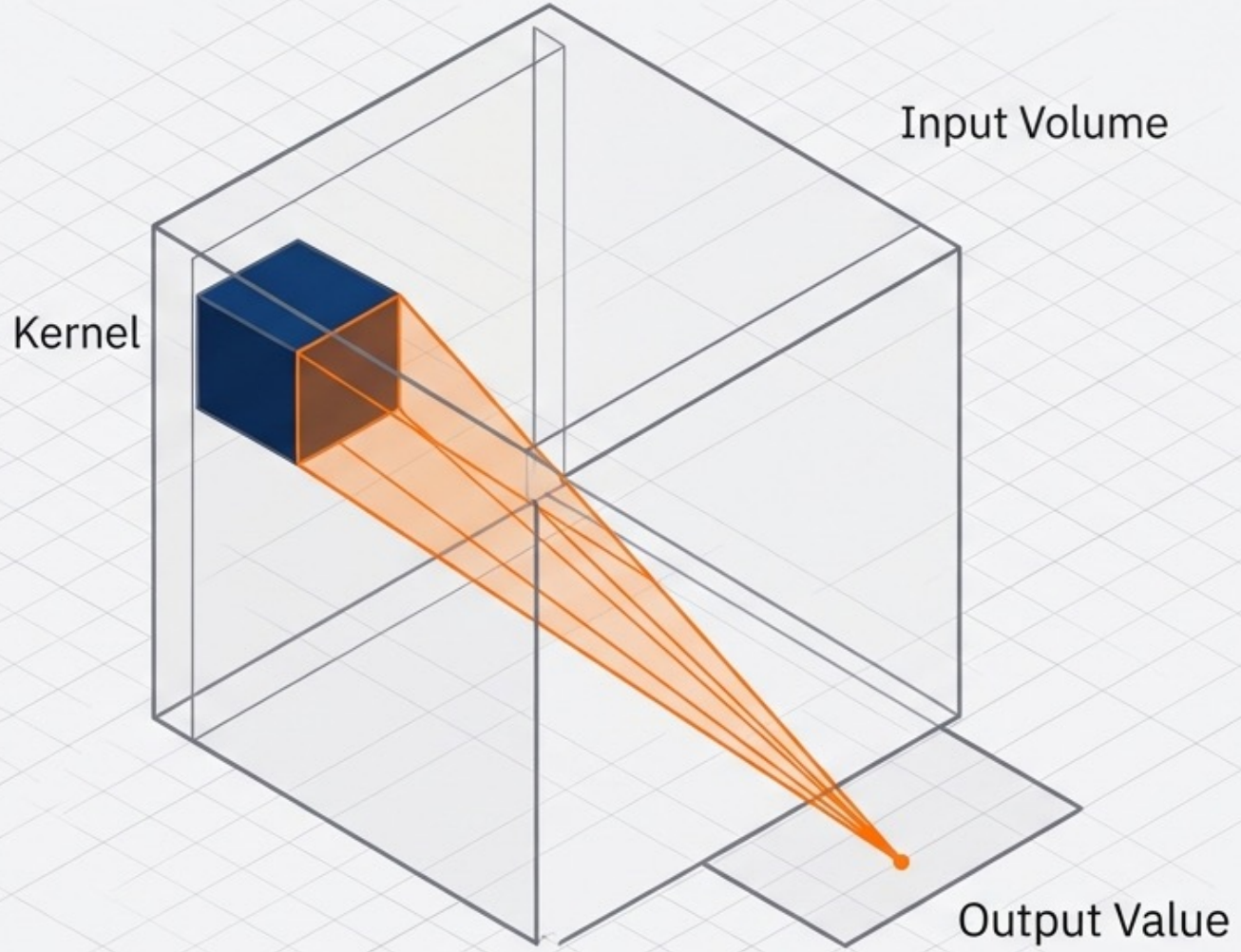
Decorrelated/Whitened Data
(No Shape Concept)



In a 1D vector, pixel (0,0) and pixel (0,1) lose their neighbor relationship.
The network loses the concept of ‘Shape’.

NotebookLM

The Solution: Local Connectivity



The Sliding Window Strategy

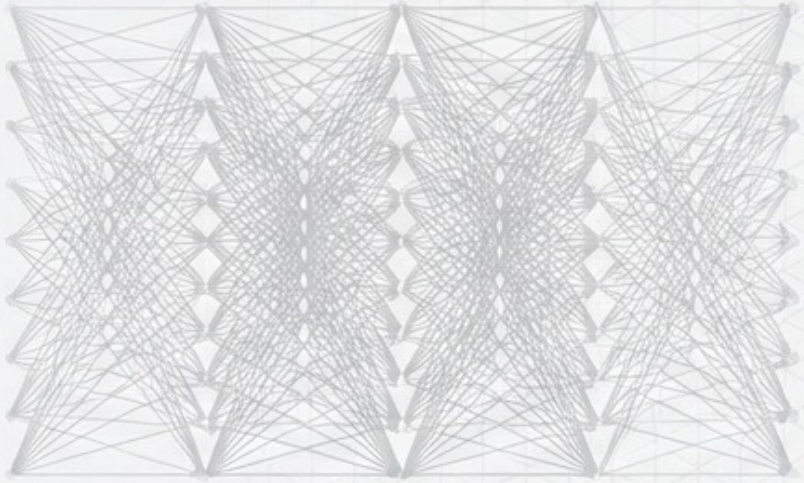
- Replaces global connections with local patches.
- The Kernel (Blue) connects only to a small sub-region.
- drastically reduces parameter count.

NotebookLM

Weight Sharing & Efficiency

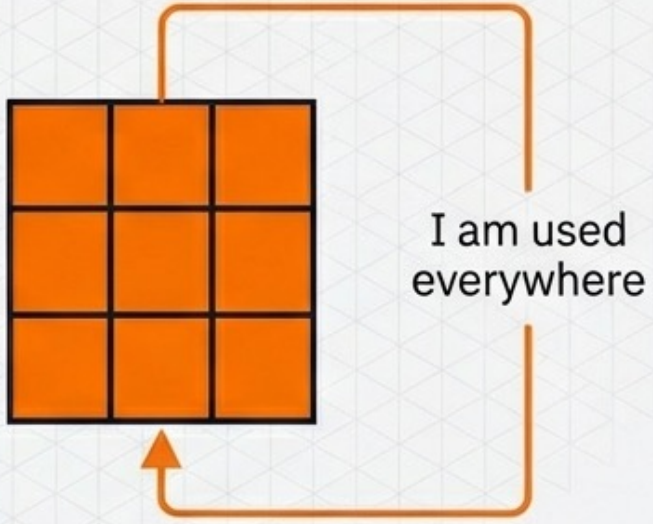
Fully Connected

`Params = Input_H * Input_W * Output_H * Output_W`



Convolution (Shared Weights)

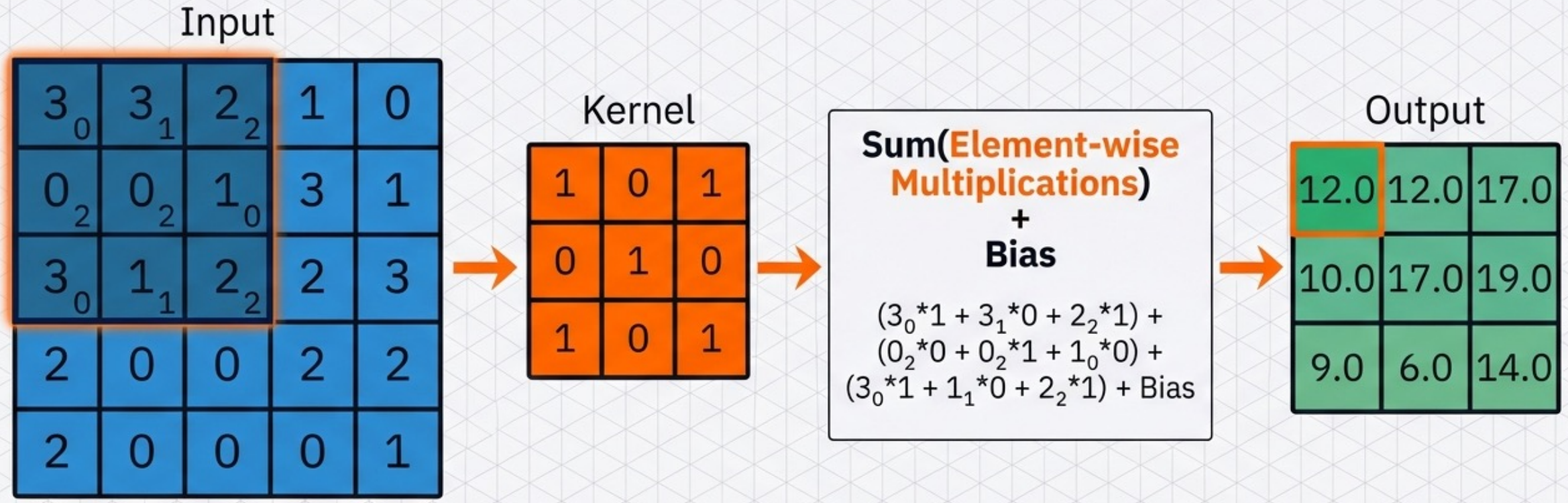
`Params = Kernel_H * Kernel_W * Channels`



Insight: If a feature is useful at `(x,y)`, it is useful at `(x+1,y)`.
We learn one filter and slide it across the entire image.

NotebookLM

Anatomy of a 2D Convolution

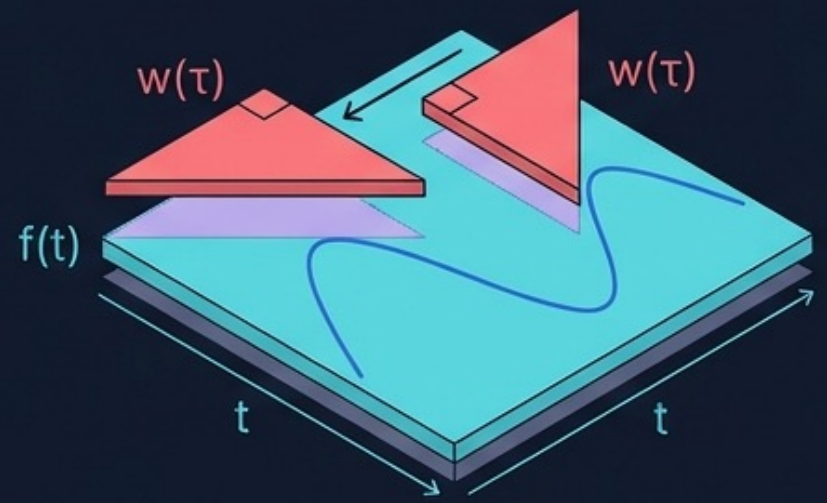


Mathematically: A dot product between the kernel weights and the local input patch.

NotebookLM

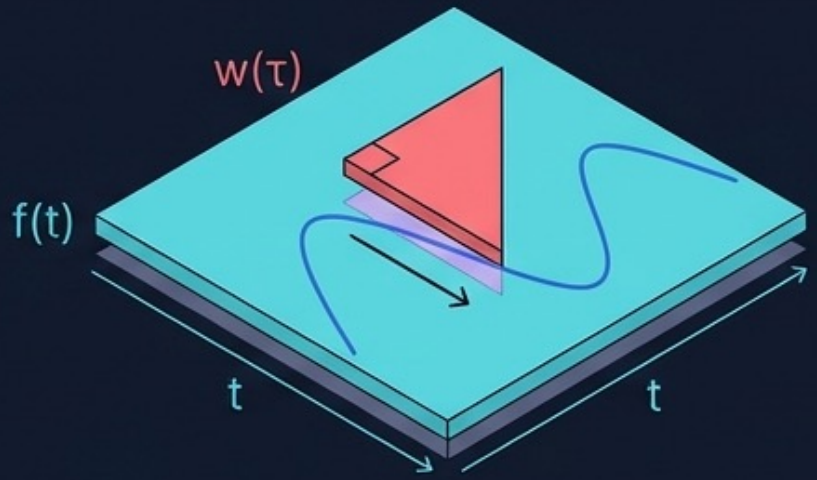
The ‘Flip’ Distinction

True Convolution



$w(t - \tau)$

Cross-Correlation (Deep Learning)

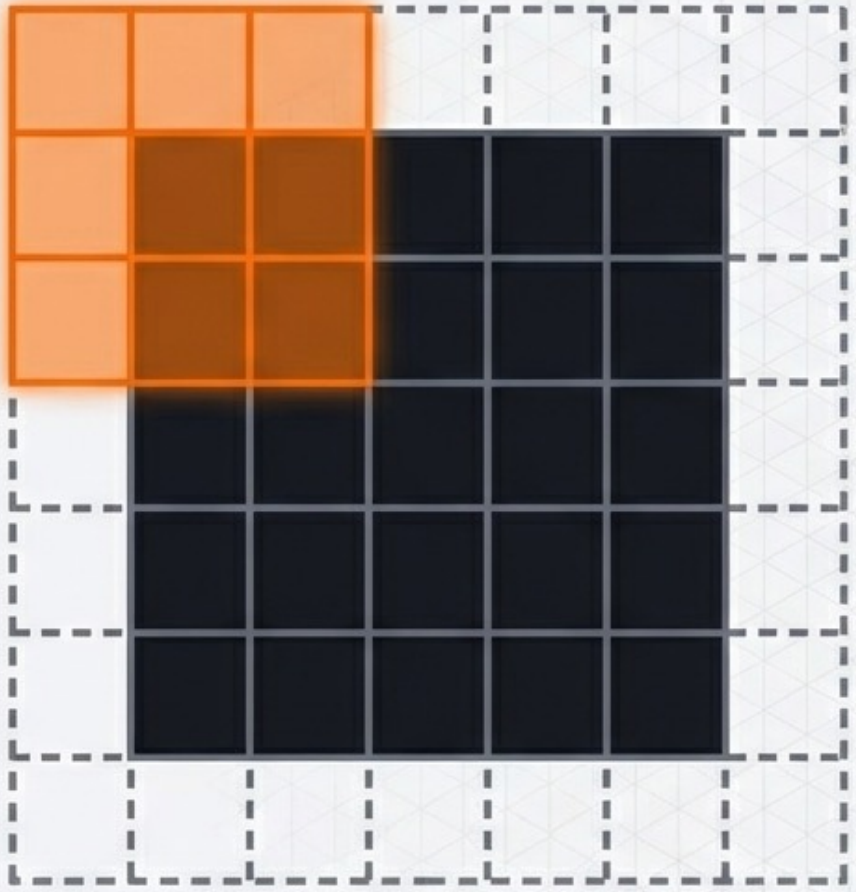


$w(t + \tau)$

Engineering Note: Deep Learning libraries (like PyTorch) technically perform Cross-Correlation but call it Convolution. Because the weights are learned from scratch, the lack of a ‘flip’ does not affect the model’s ability to learn features.

NotebookLM

Controlling Geometry: Padding



Problem: Without padding, the image shrinks with every layer.

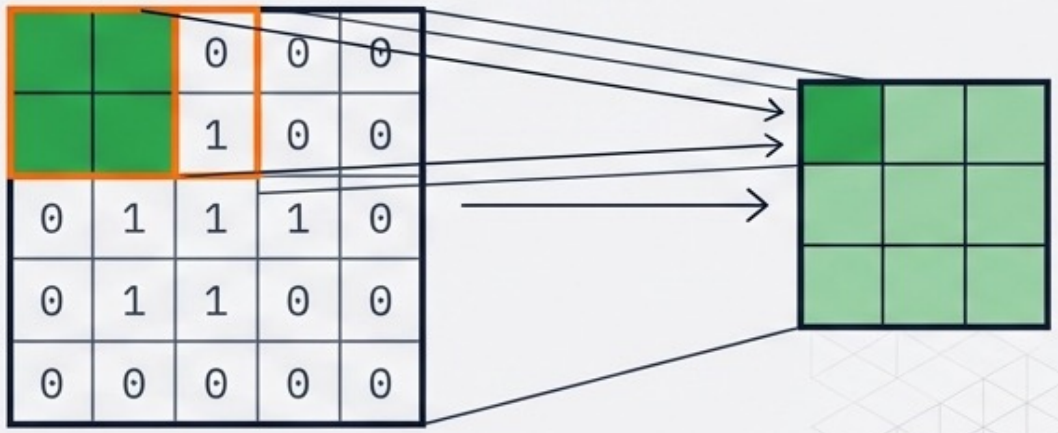
Valid Padding: No padding. Output < Input.

Same Padding: Add borders (zeros) to maintain dimensions. Output = Input.

NotebookLM

Controlling Resolution: Striding

Stride = 1



Stride = 2



- **Stride:** The step size of the **sliding window.**
- **Stride > 1** results in **downsampling** (compression) **and reduced** computation.

NotebookLM

Calculating Output Dimensions

$$H_{out} = \text{floor}((H_{in} + 2*P - D*(K-1) - 1) / S + 1)$$

P: Padding
(Pixels added to border)

K: Kernel Size
(Size of the window)

S: Stride
(Step size)

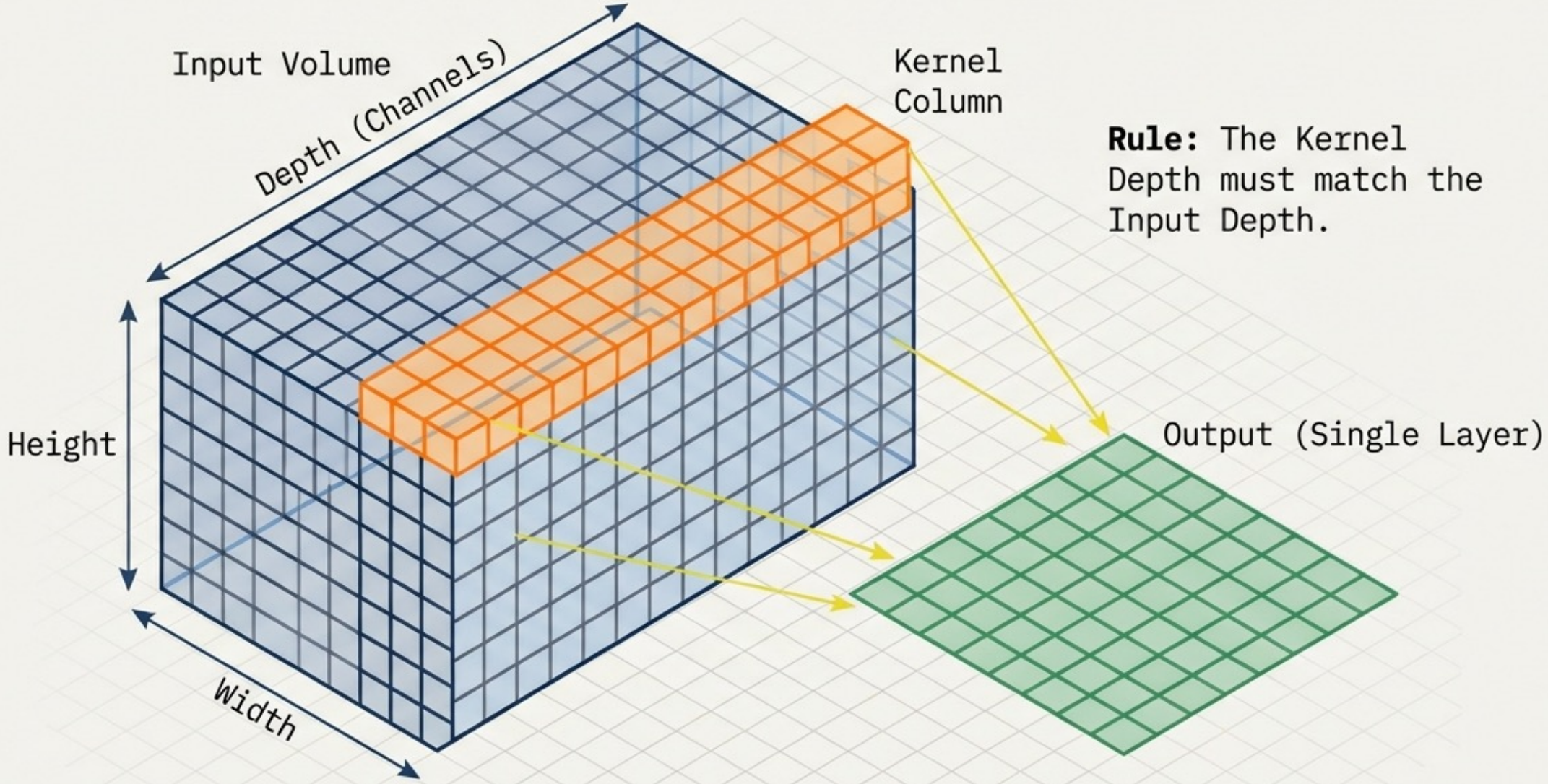
H_out =
3 thin hair-lines (0.5pt)

D: Dilation
(Spacing between kernel elements)

```
Example: Input 32x32, Kernel 3x3, Stride 1, Padding 0.
Result: (32 + 0 - 2) / 1 + 1 = 31 -> Output 31x31
```

NotebookLM

The Third Dimension: Volume

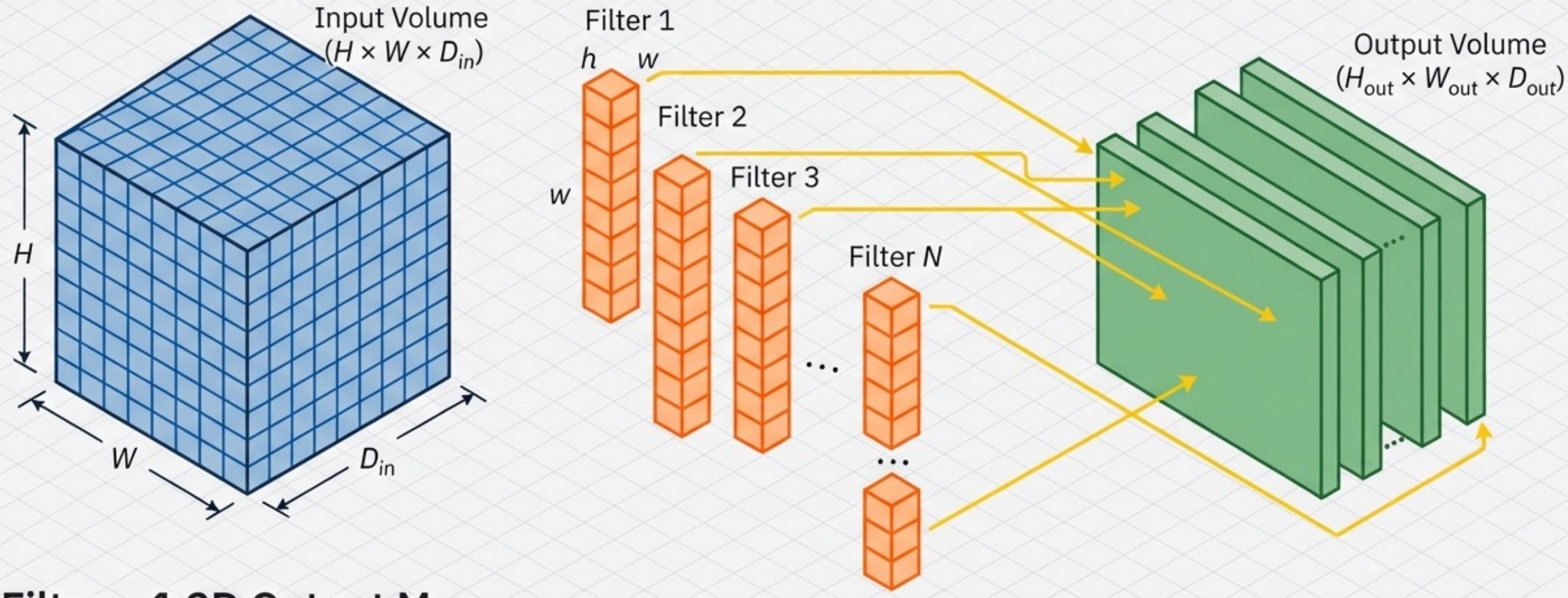


Rule: The Kernel Depth must match the Input Depth.

NotebookLM

Modern Engineering Blueprint

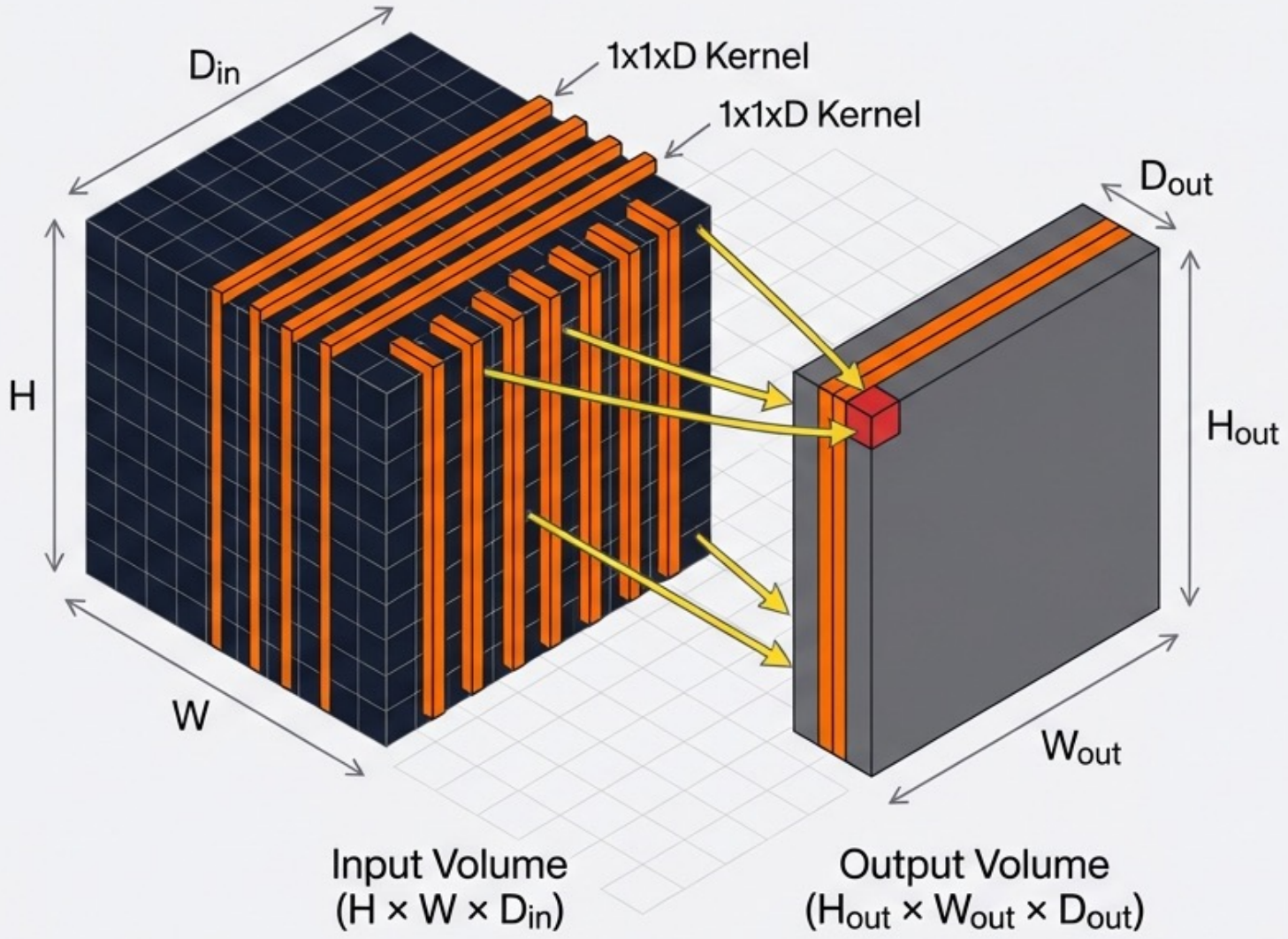
Generating Multi-Channel Features



1 Filter = 1 2D Output Map
N Filters = N Channels Output Volume

NotebookLM

The Special Case: 1x1 Convolution



Structure: A kernel of size 1x1xD.

Function: Does not see spatial neighbors. Only mixes information across channels.

Use Case: Dimensionality reduction (bottleneck) or expansion.

NotebookLM

Deciphering Feature Maps: What & Where



Channel #55 Response

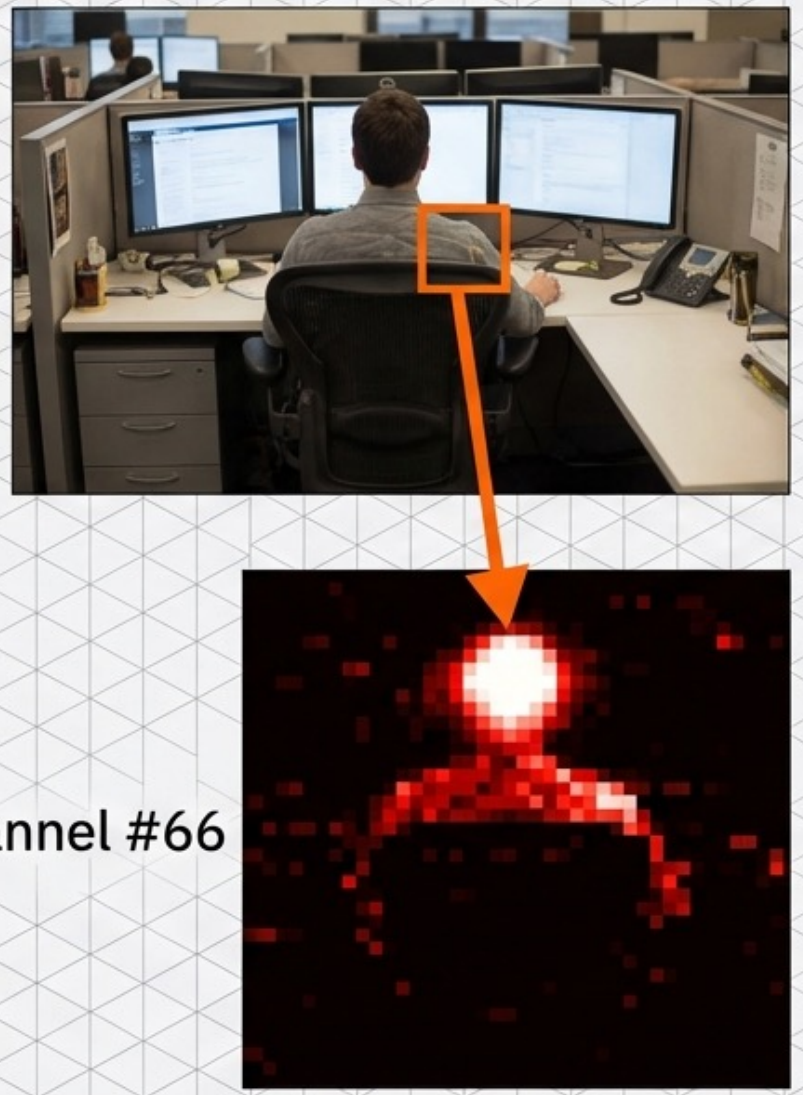
WHAT: Circle-shaped texture detector (Tire).

WHERE: The high activation (white pixel) corresponds exactly to the tire's location in the image.

A feature map is a map of ***presence***.

NotebookLM

Learned Representations



Channel #66

Channel #66 Response



WHAT: Lambda (Λ) shaped contours (shoulders/underarms).

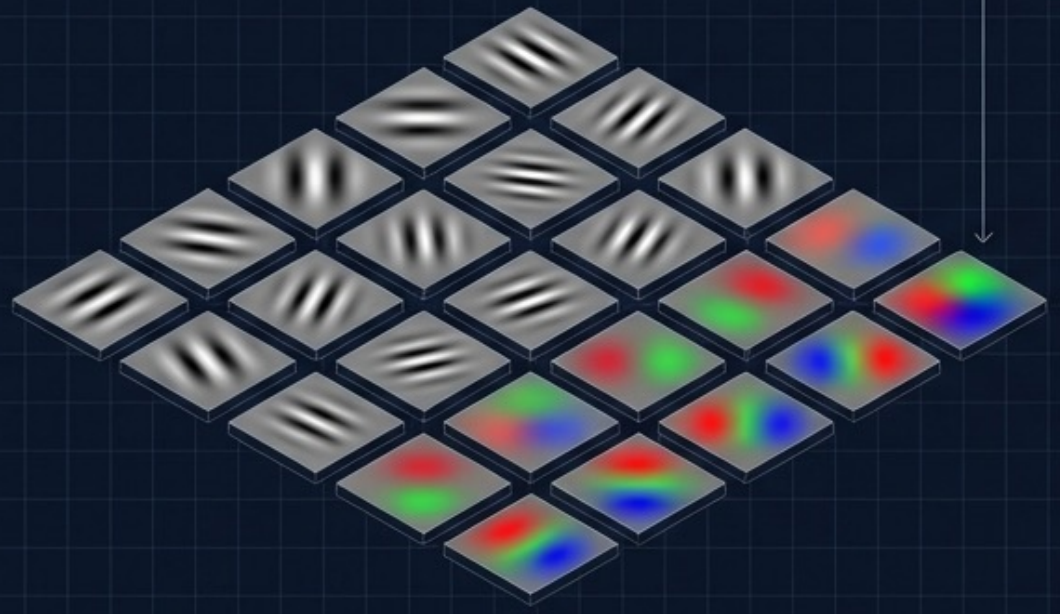
Deep networks build complex semantic understanding (Person) from simple geometric detections (Lambda shape).

NotebookLM

Inside the Black Box

Layer 1 Weights (Retina)

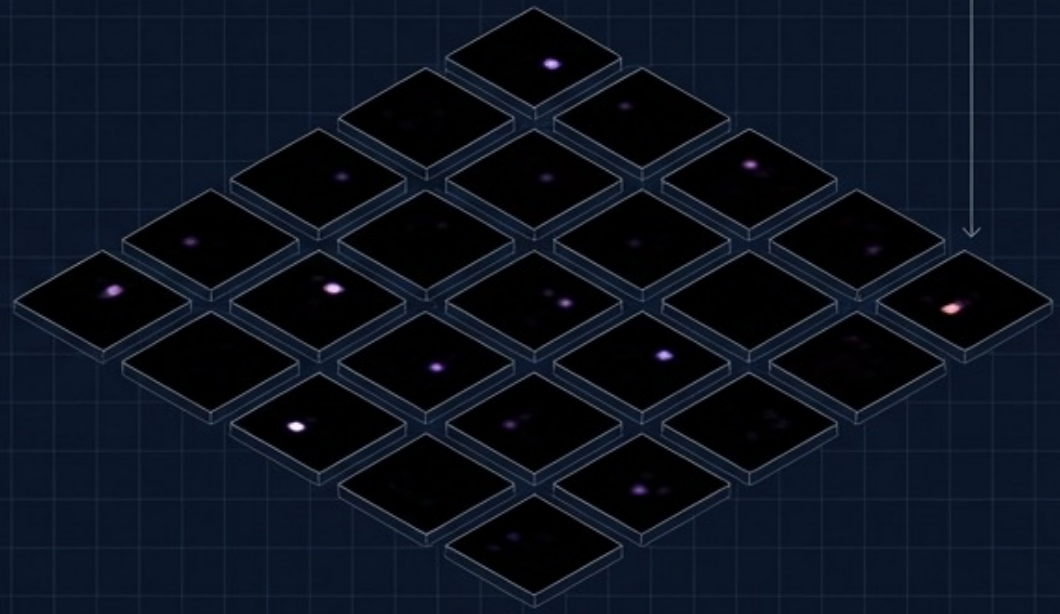
JetBrains Mono Medium



Smooth, low-frequency edge & color detection. Indicates a converged network.

Layer 5 Activations (Brain)

JetBrains Mono Medium

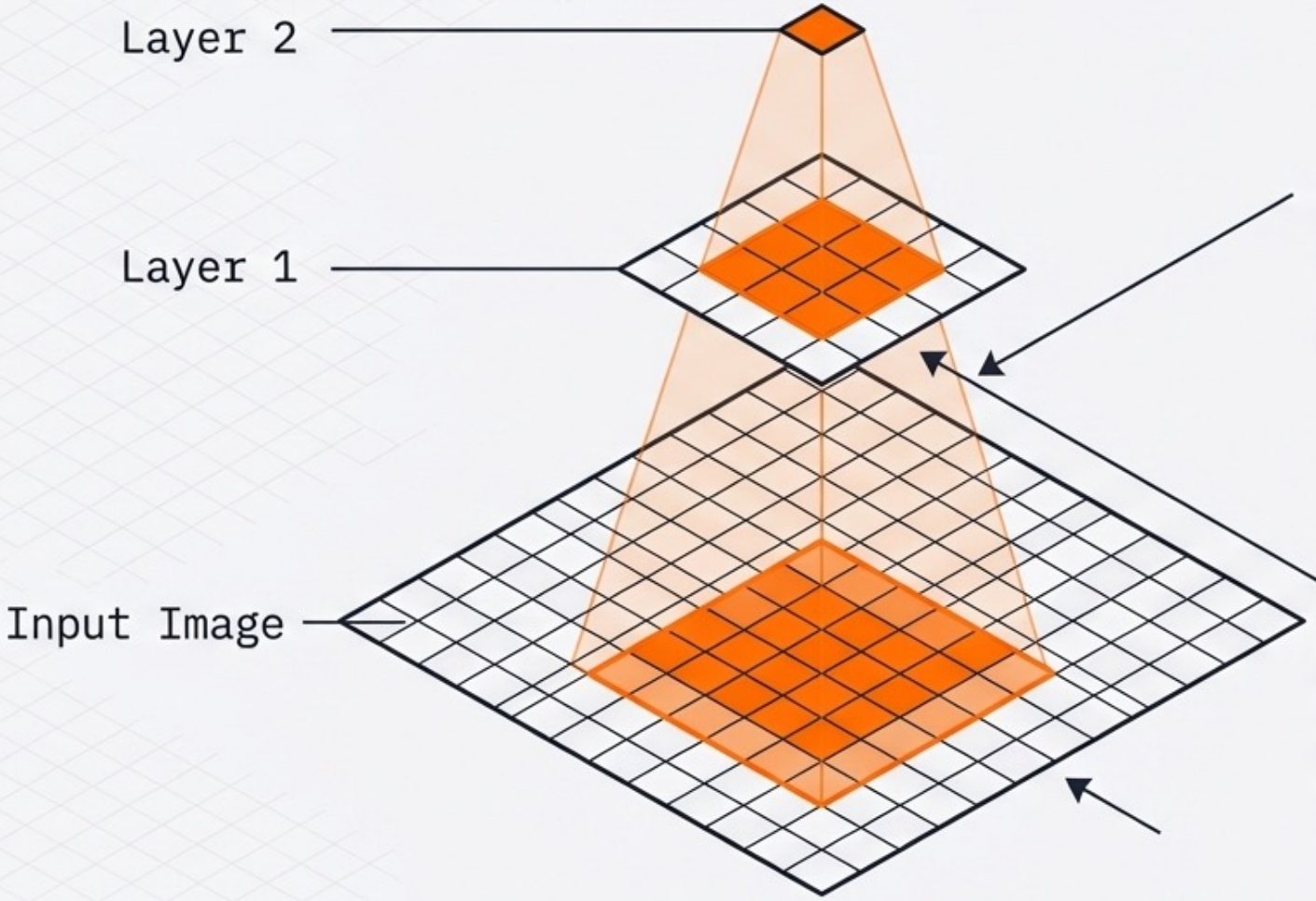


Sparse, high-level semantic activations.

Visualizations based on trained AlexNet layers.

NotebookLM

The Receptive Field



Receptive Field: The region of the original image that a specific neuron “sees”.

As we go deeper, the Receptive Field expands, allowing the network to understand global context.

NotebookLM

The Modern Architecture



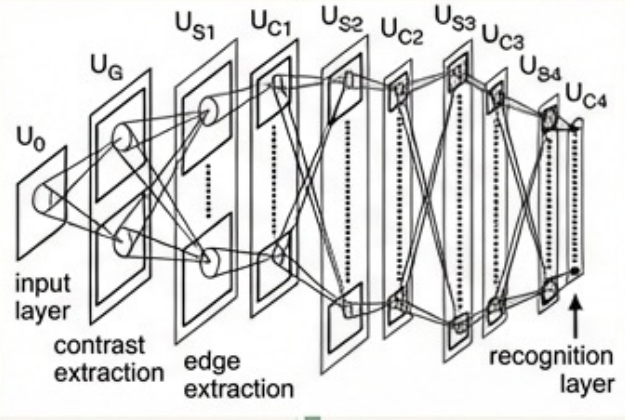
End-to-End Learning Paradigm:

- We define the skeleton (Architecture).
- The data trains the brain (Weights).
- No more hand-crafted features.

IBM Plex Mono

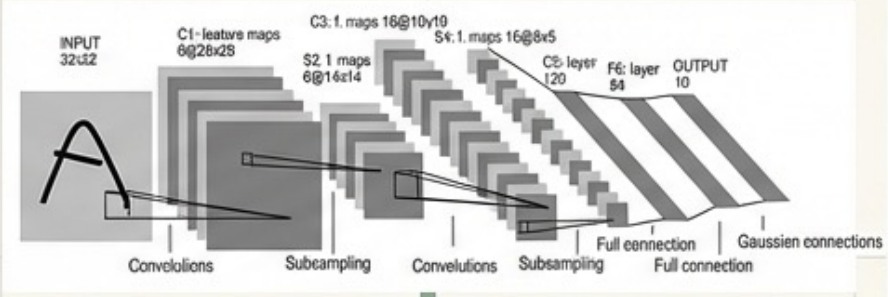
NotebookLM

The Early Pioneers of Vision



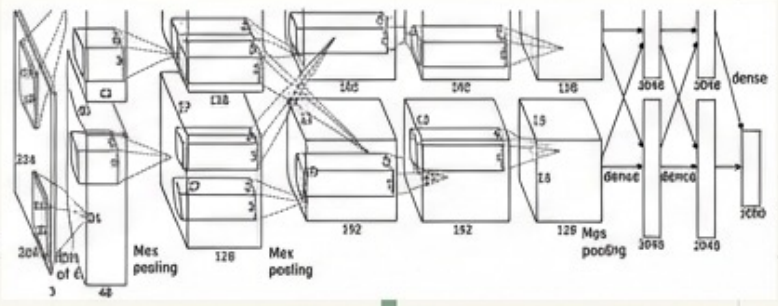
1980: Neocognitron

Fukushima introduces hierarchical multilayered neural networks with feature-extracting S-cells and C-cells.



1998: LeNet5

LeCun applies gradient-based learning, introducing average pooling and tanh nonlinearities for digit recognition.



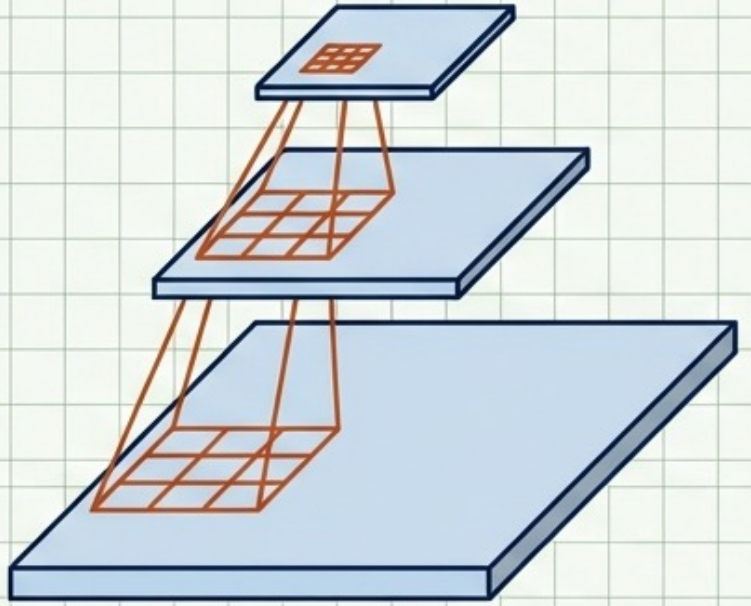
2012: AlexNet

Krizhevsky scales up. 60M parameters, max pooling, ReLU, and the critical introduction of GPU-accelerated training.

NotebookLM

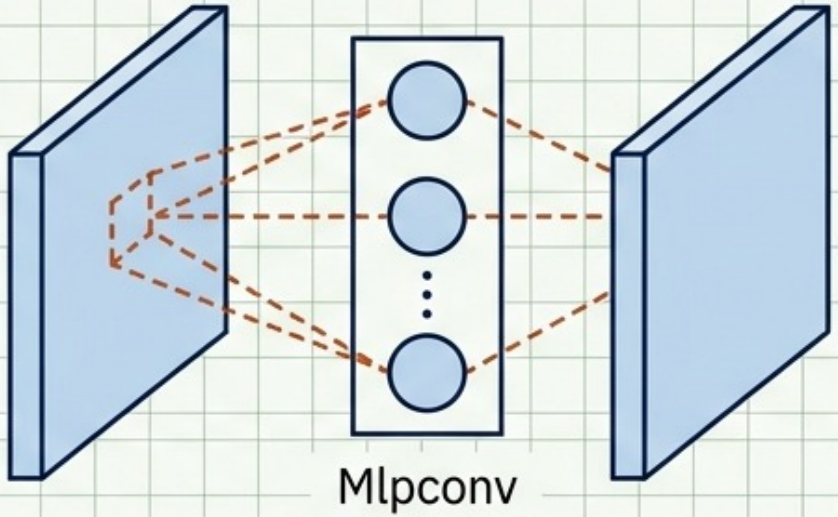
Standardizing the Micro-Blocks

VGG16 (2014)



- Replaced large, expensive receptive fields (7x7) with successive layers of 3x3 convolutions.
- Efficiency breakthrough: Three 3x3 layers require drastically fewer weights than one 7x7 layer.

Network in Network (2014)



- Introduced the highly versatile 1x1 convolution (Mlpconv layer) to easily control and reduce dimensionality before expensive operations.

NotebookLM

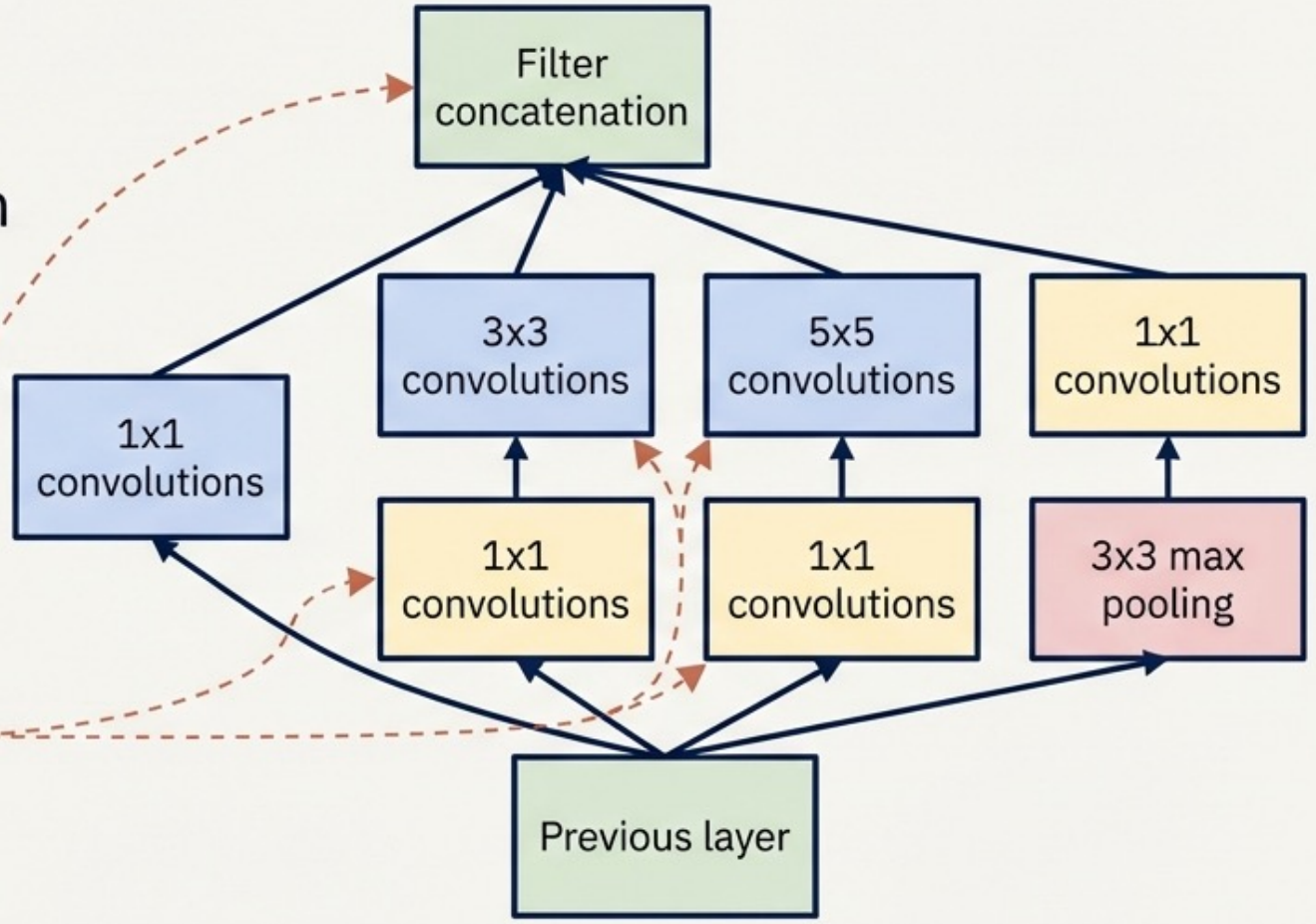
The Golden Heuristic: Split, Transform, Aggregate

GoogLeNet / Inception v1 (2015)

Breakthrough: Instead of choosing a single filter size, use parallel paths with different receptive fields to capture sparse correlation patterns.

The Heuristic Core:

1. **Split** the input.
2. **Transform** via parallel 1x1, 3x3, and 5x5 convolutions.
3. **Aggregate** via filter concatenation.



NotebookLM

The Deep Optimization Crisis

The Bottleneck

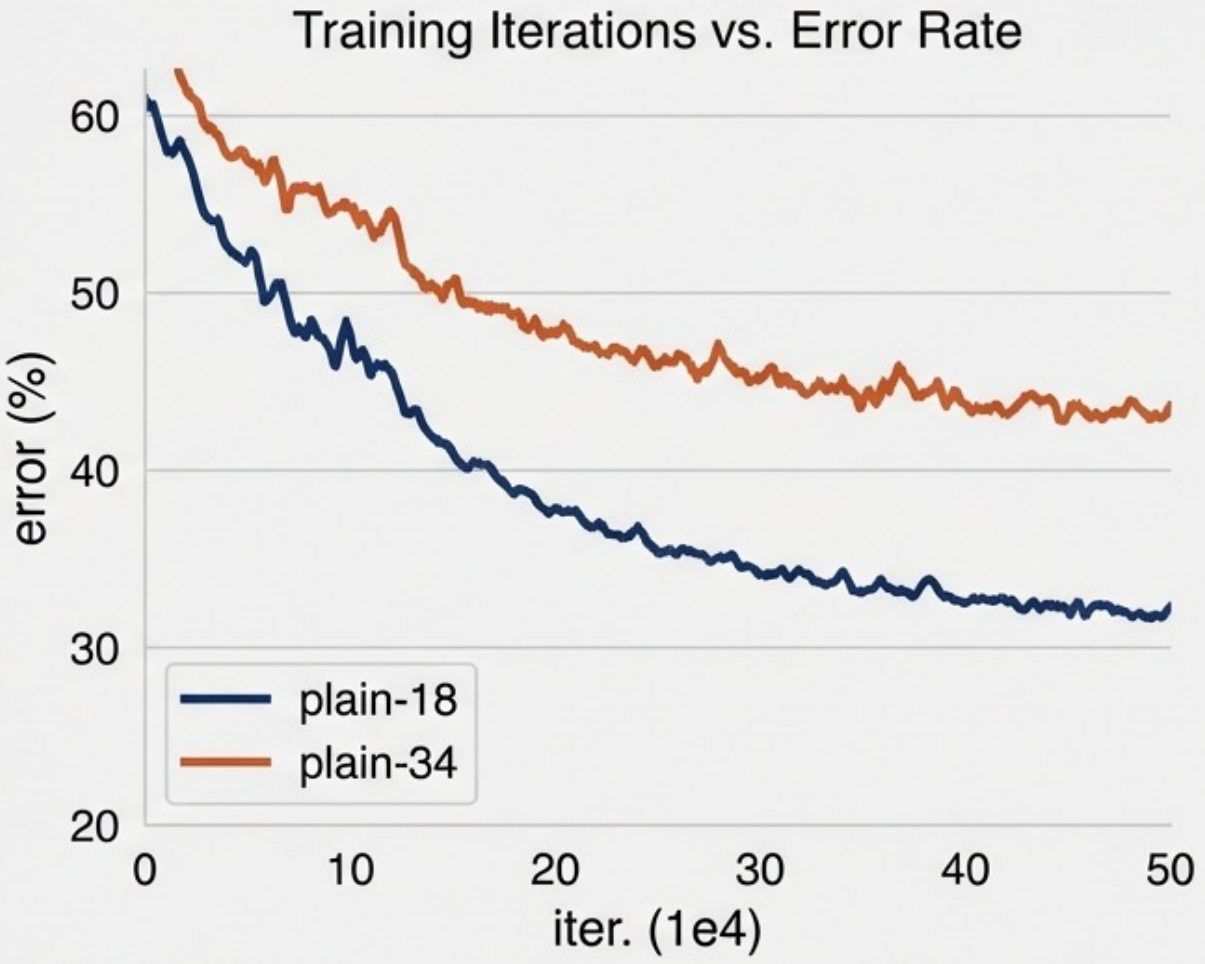
We need to go deeper to gain representational power.

The Wall

Adding more layers to a plain network actually degrades performance.

The Cause

The vanishing/exploding gradient problem prevents stochastic gradient descent from optimizing infinitely deep models.



NotebookLM

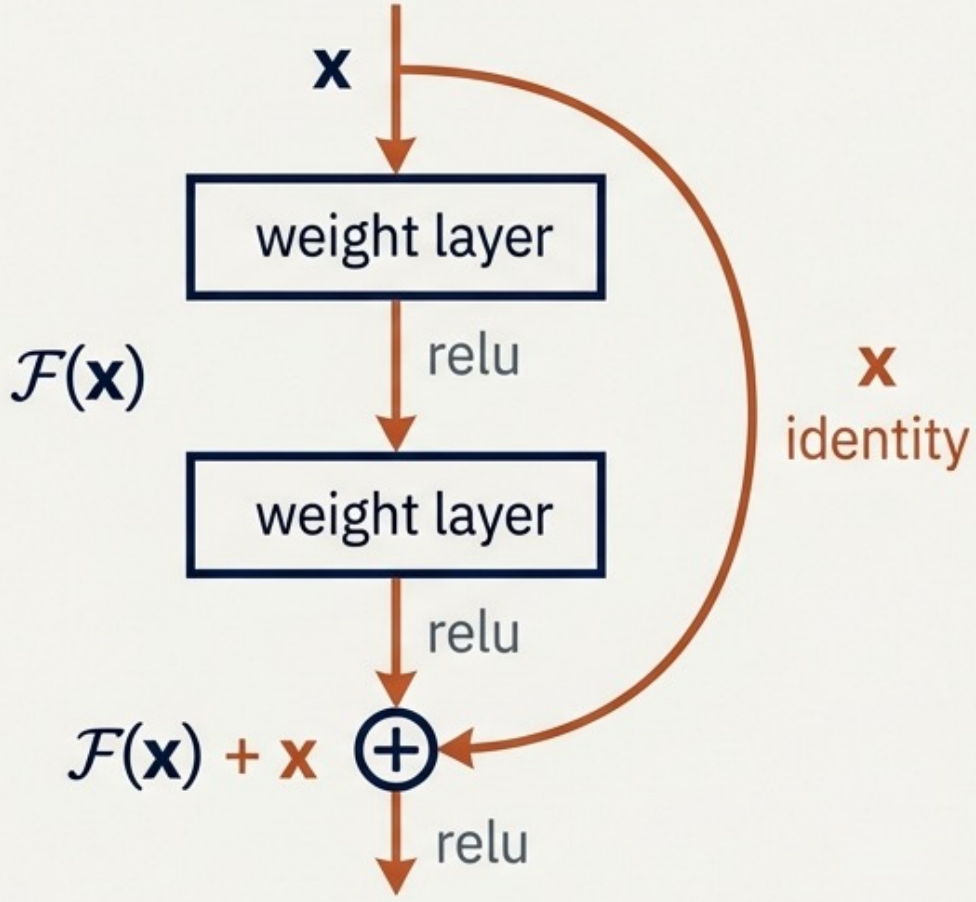
The Residual Revolution

ResNet (2016)

Kaiming He et al. fundamentally alter network topology.

The Breakthrough

The **Identity Mapping / Skip Connection**.



The Math

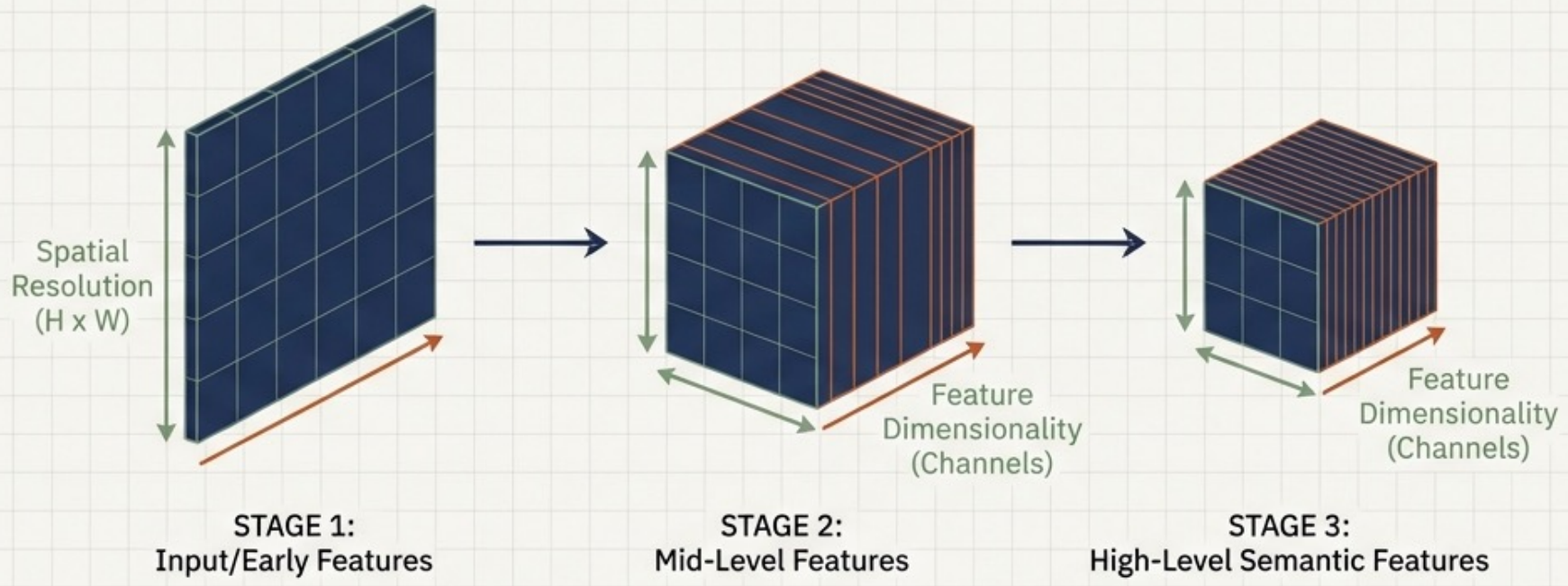
Instead of forcing layers to learn a completely new mapping, they learn a **residual function**. The output becomes **F(x) + x**.

The Result

Gradients **flow safely** through the identity path, allowing networks to scale to **hundreds of layers**.

NotebookLM

The Stage-Wise Architectural Blueprint



The Macro-Structure

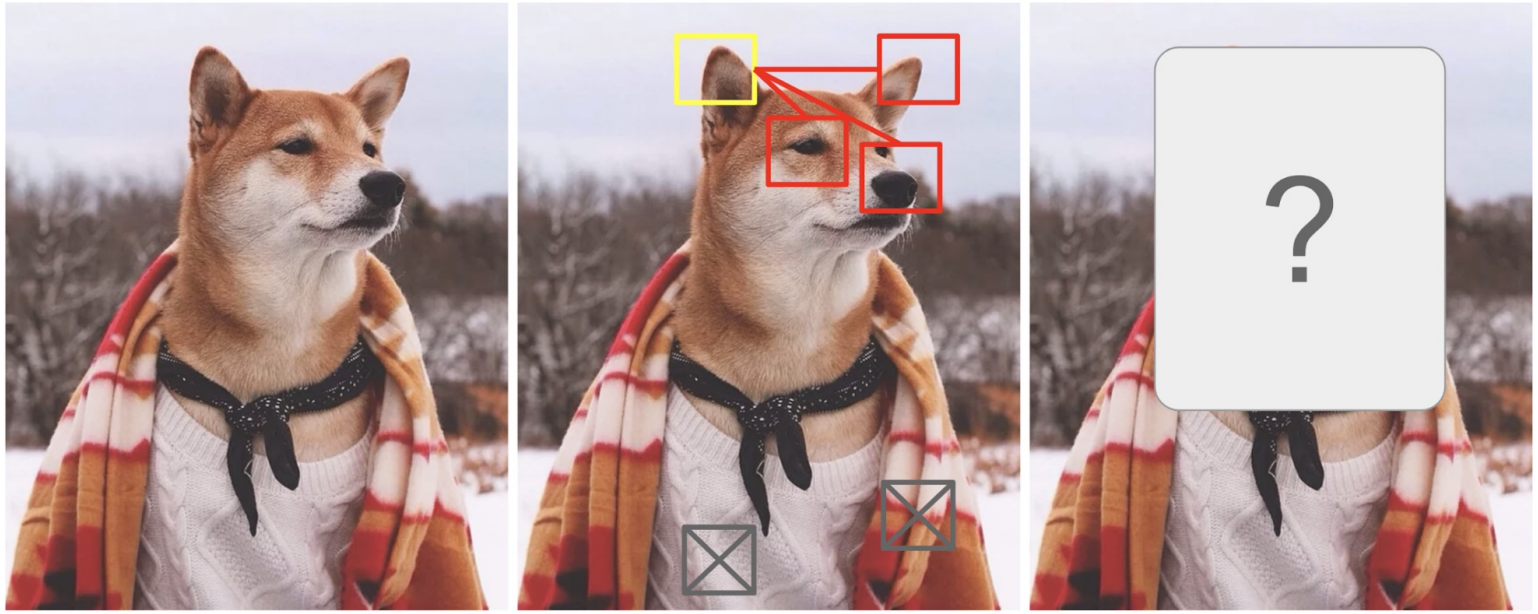
Modern vision models navigate the search space by adopting a predictable, stage-wise building block design.

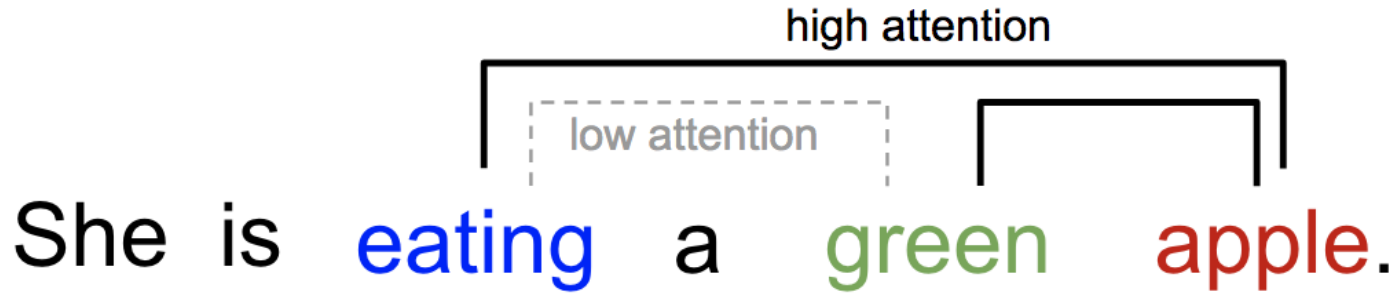
The Rule of Thumb

As we progress deeper into the network, spatial resolution strictly decreases (height/width shrink), while feature dimensionality strictly increases (channels expand).

NotebookLM

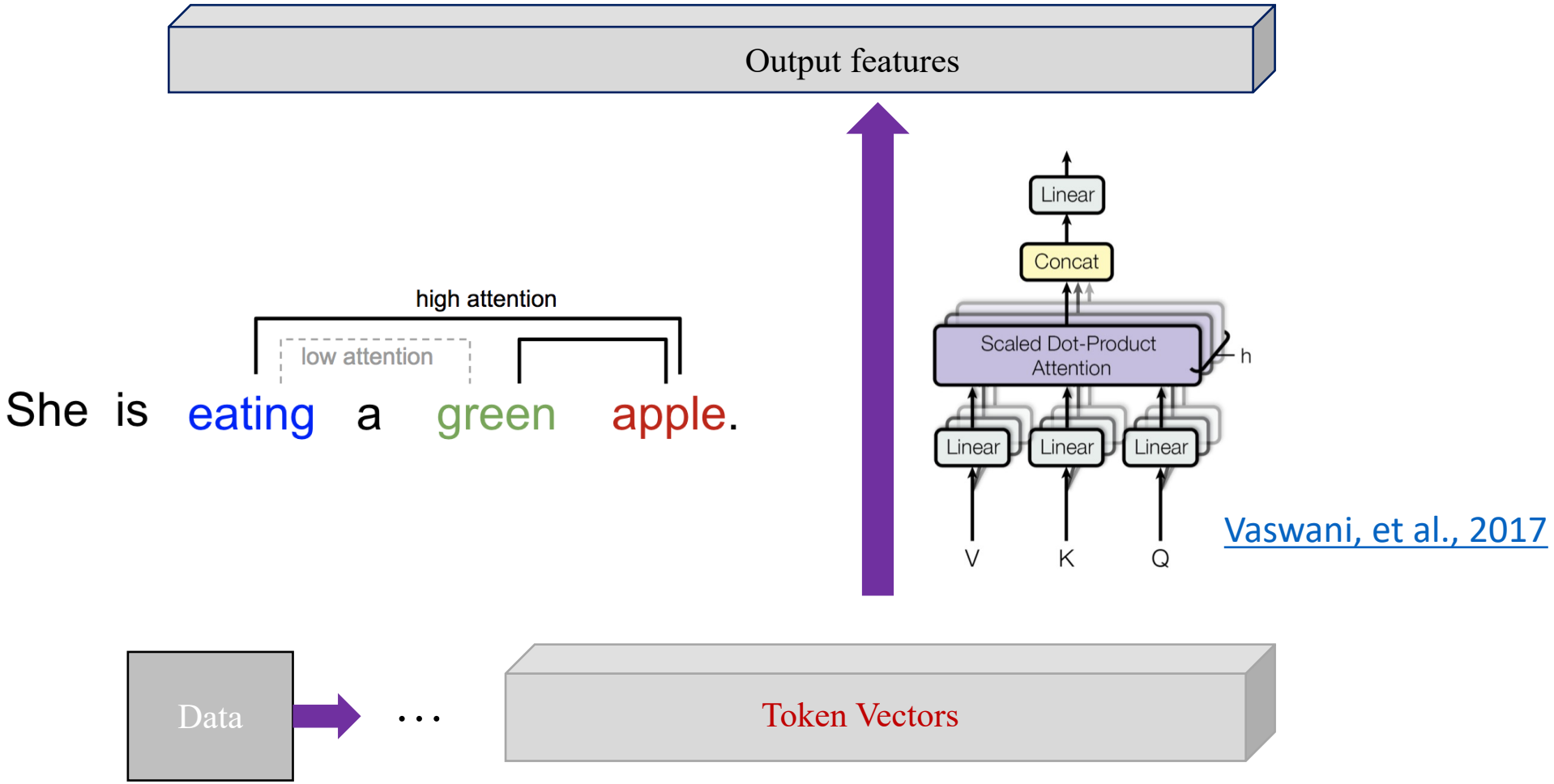
Attention? Attention!





https://lilianweng.github.io/posts/2018-06-24-attention/

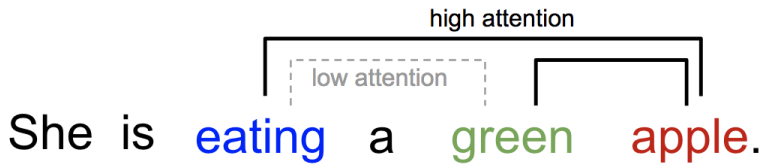
Modeling Attention



Vaswani, et al., 2017

Transformers' Core: Attention

- What to process/encode/extract and how? – Modeling pairwise relationships between tokens in a deep manner



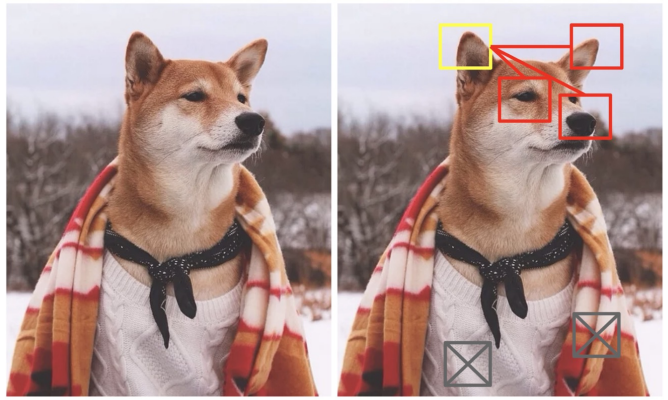
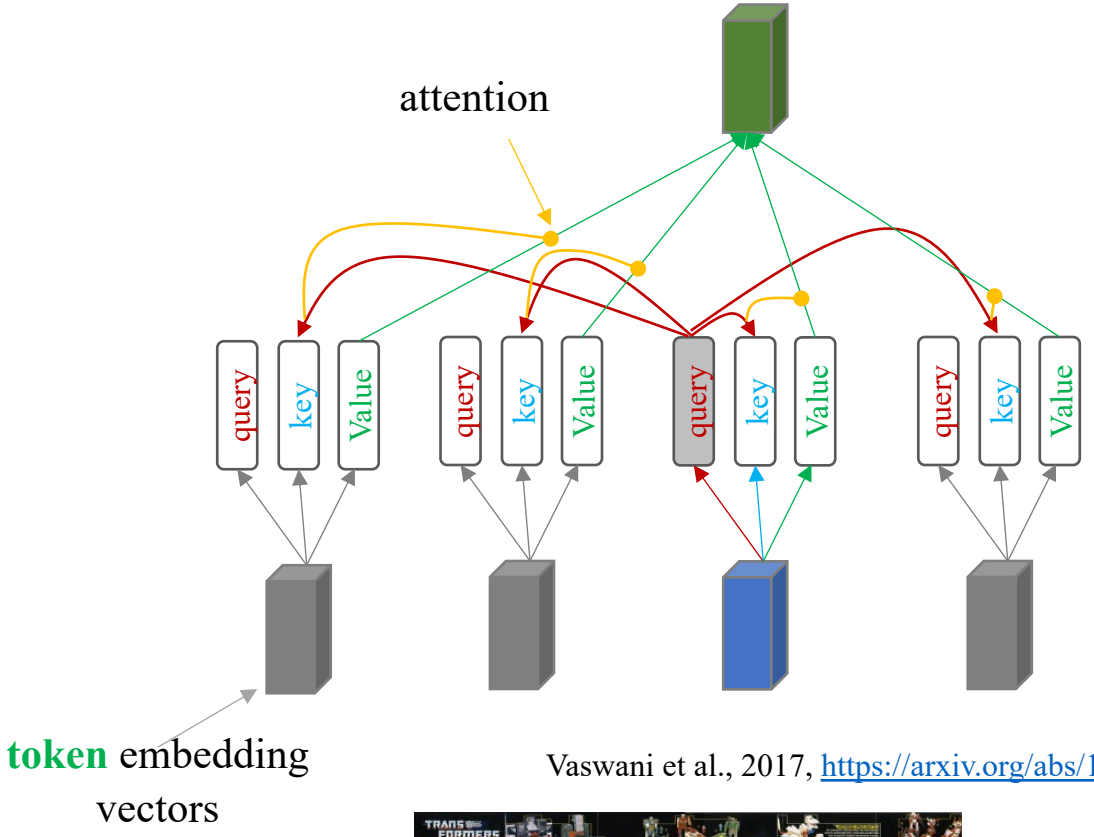


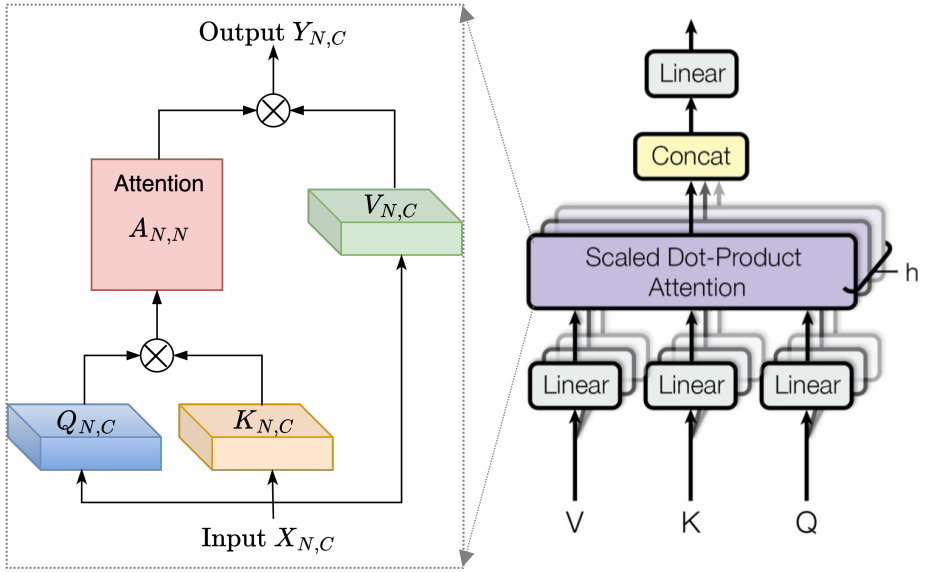
Figure credit: https://lilianweng.github.io/posts/2018-06-24-attention/



Vaswani et al., 2017, https://arxiv.org/abs/1706.03762



Tokenization + Token Mixing + Channel Mixing



Input: $X_{N\times d}$, N tokens each of which is embedded in a D-dim space

$$Q_{N\times C} = X_{N\times C}W^{Q}_{C\times C} = Q_{H\times N\times d}$$
$$K_{N\times C} = X_{N\times C}W^{K}_{C\times C} = K_{H\times N\times d}$$
$$V_{N\times C} = X_{N\times C}W^{V}_{C\times C} = V_{H\times N\times d}$$

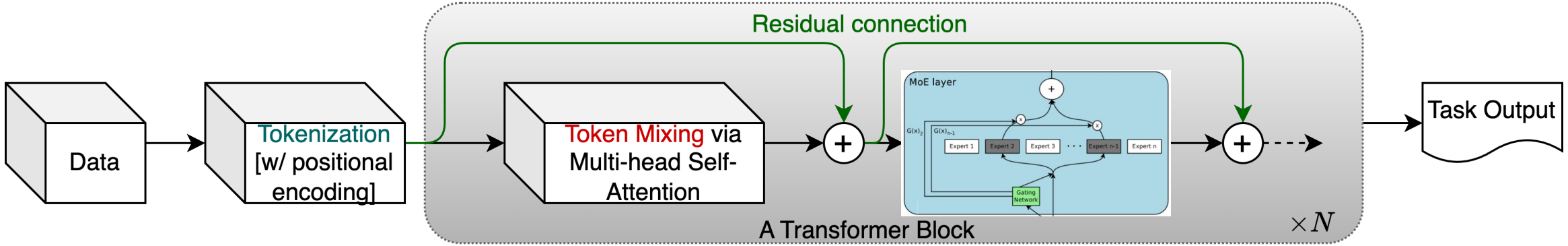
Reshape
Multi-head / subspace
$C = Hd$

Output:

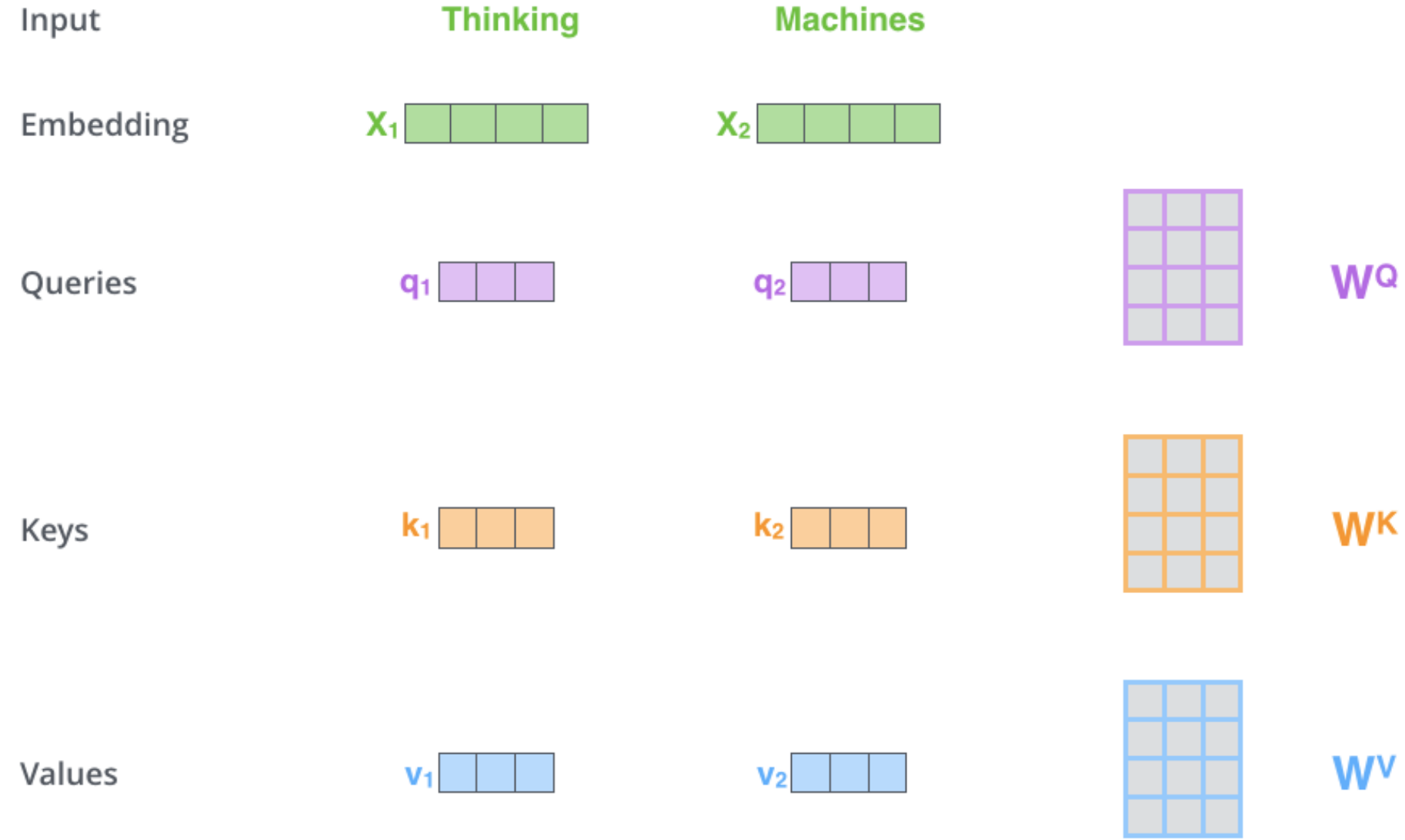
One head: $Y_{N\times d} = A_{N\times N}V_{N\times d}$

where $A_{N\times N} = softmax\left(\frac{Q_{N\times d}K^{T}_{N\times d}}{\sqrt{d}}\right)$

Multi-head: $Y_{N\times C} = \left(Y^{1}_{N\times d}, Y^{2}_{N\times d}, \cdots, Y^{H}_{N\times d}\right)W^{Proj}_{C\times C}$,

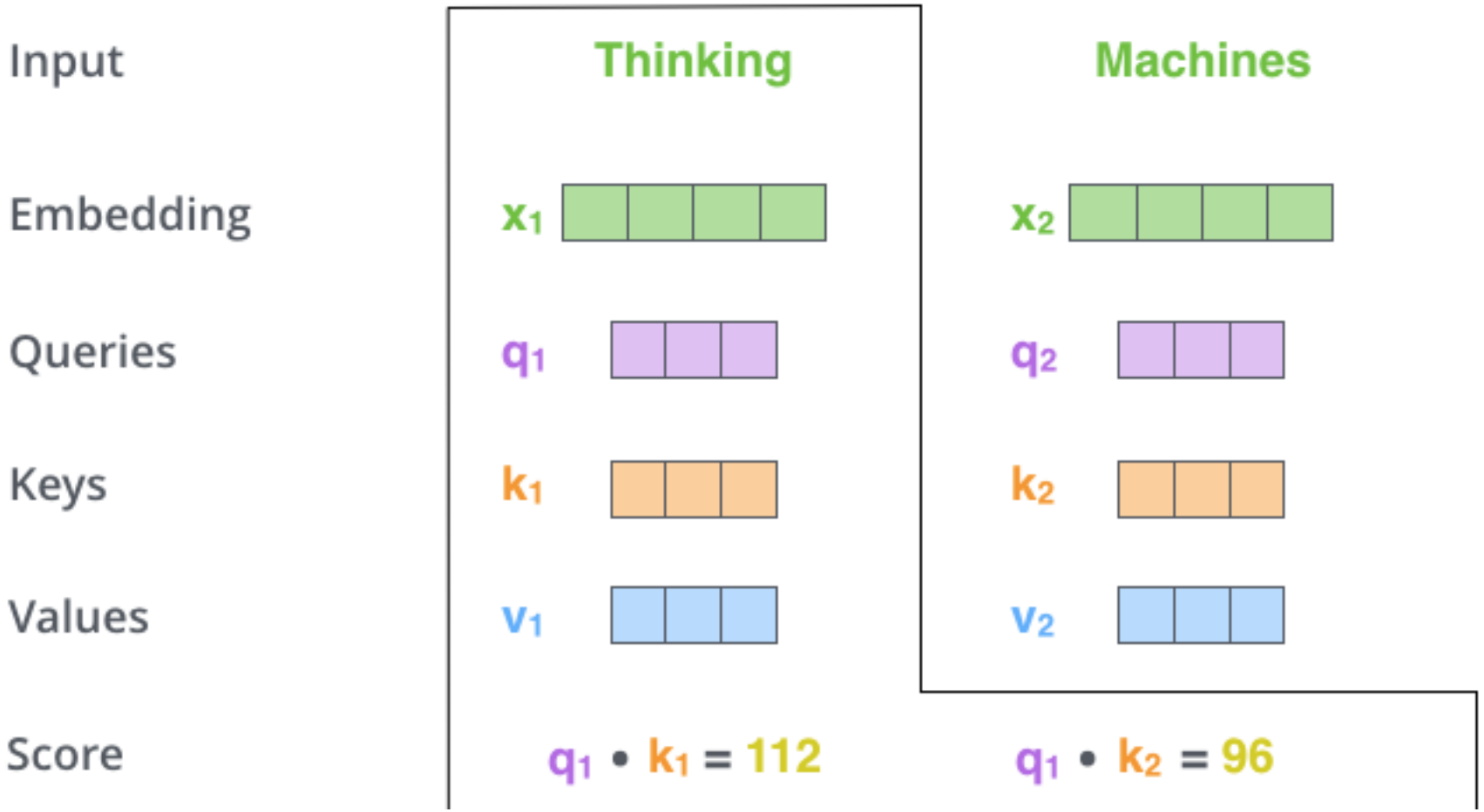


Transformer: A running example



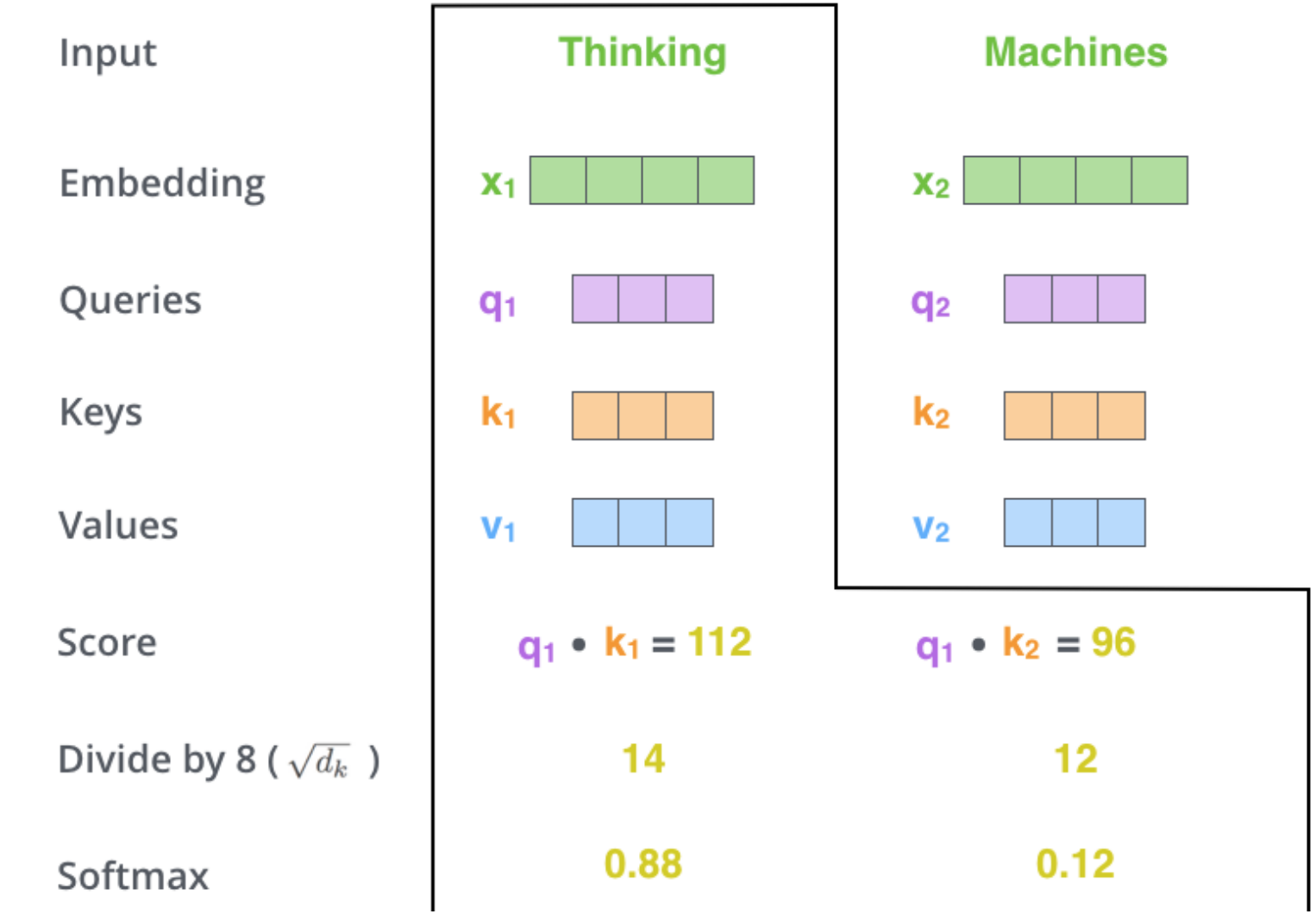
https://jalammar.github.io/illustrated-transformer/

Transformer: A running example



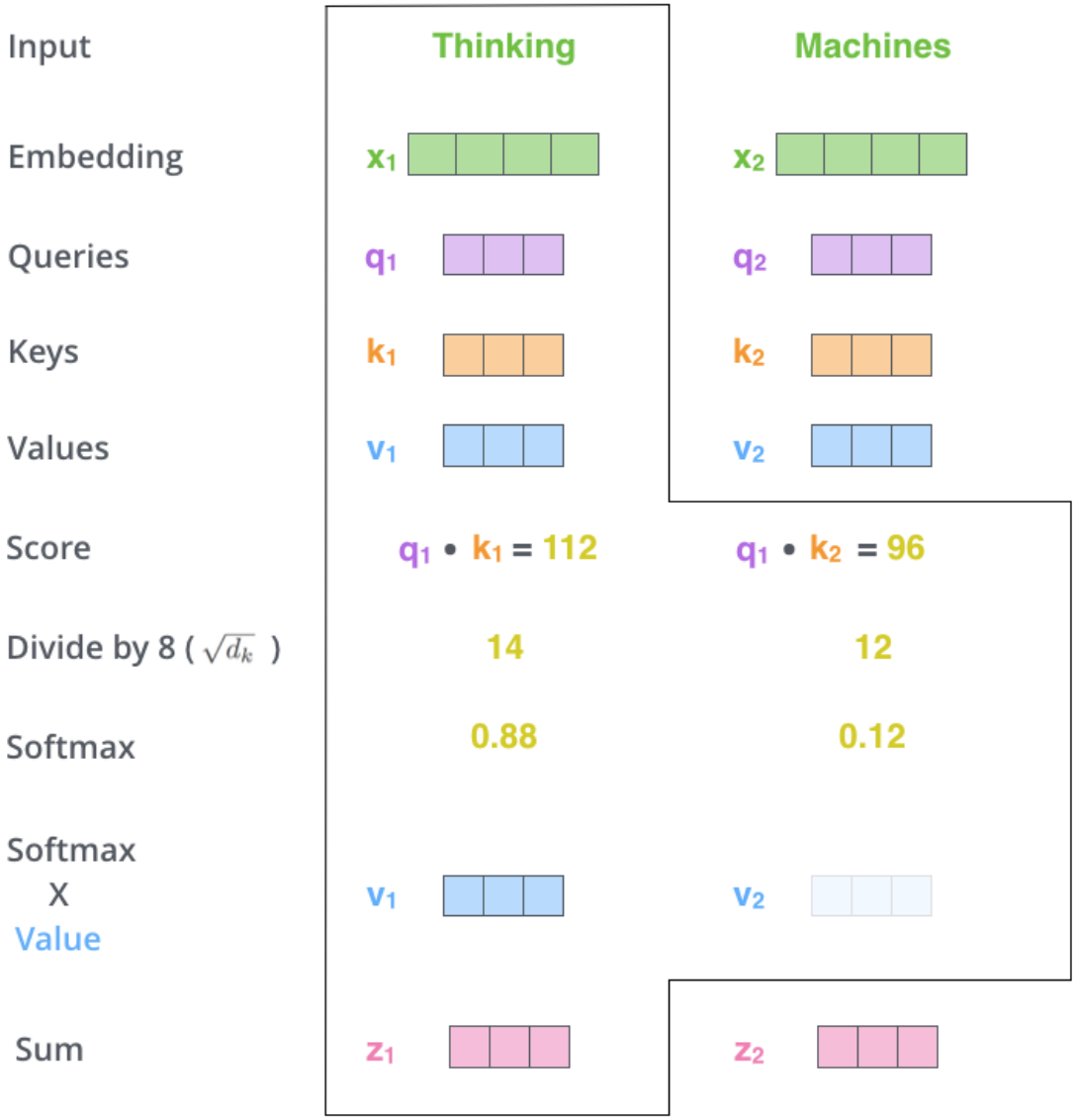
https://jalammar.github.io/illustrated-transformer/

Transformer: A running example



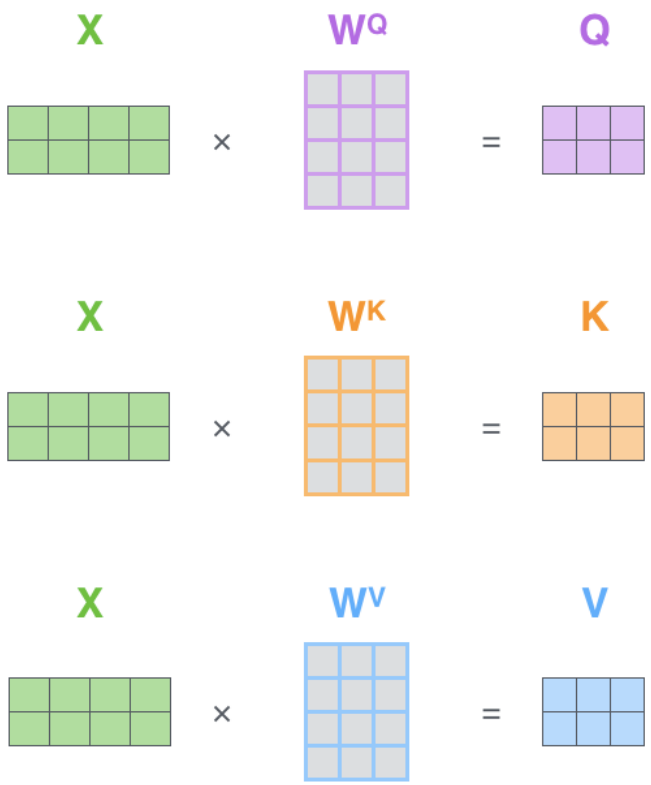
https://jalammar.github.io/illustrated-transformer/

Transformer: A running example



https://jalammar.github.io/illustrated-transformer/

Transformer: A running example

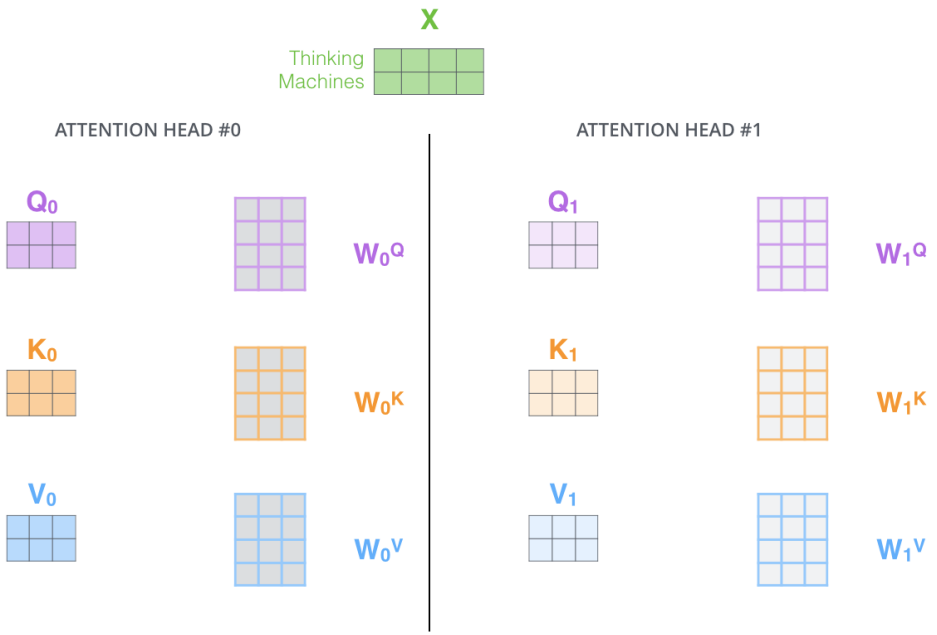


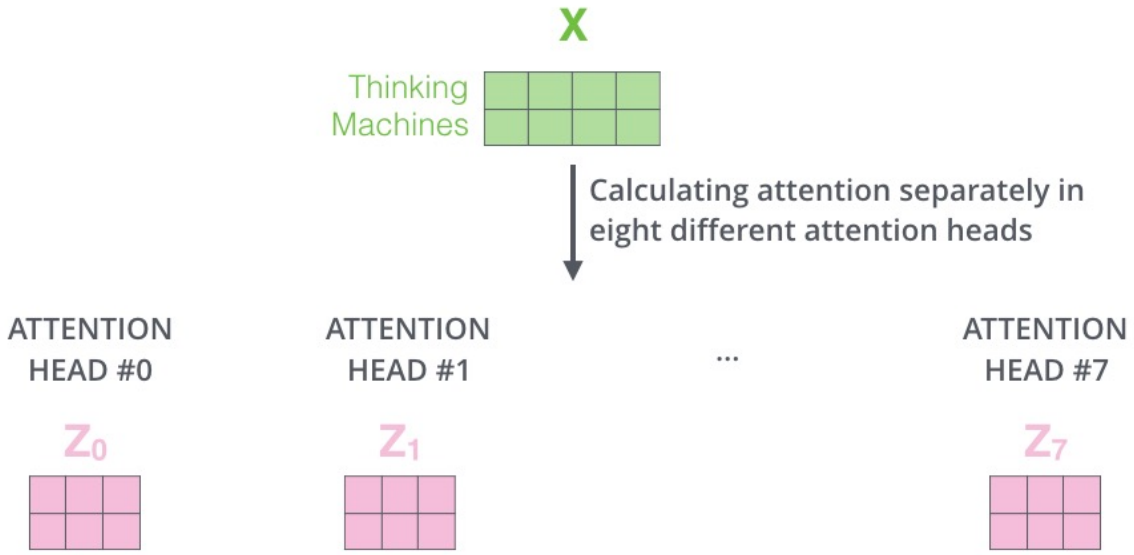
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

https://jalammar.github.io/illustrated-transformer/

Transformer: A running example

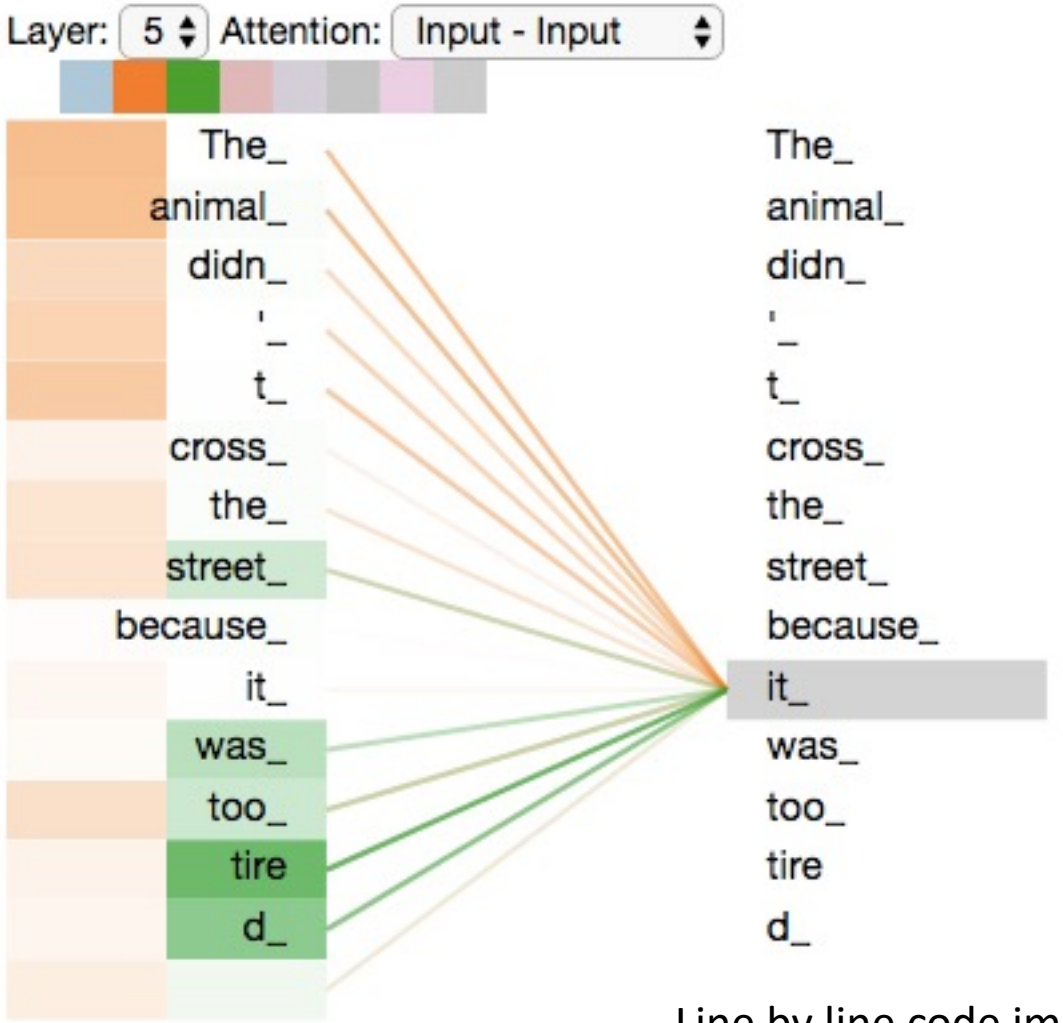
- Multi-head formulation: attention in different sub-spaces





https://jalammar.github.io/illustrated-transformer/

Transformer: A running example

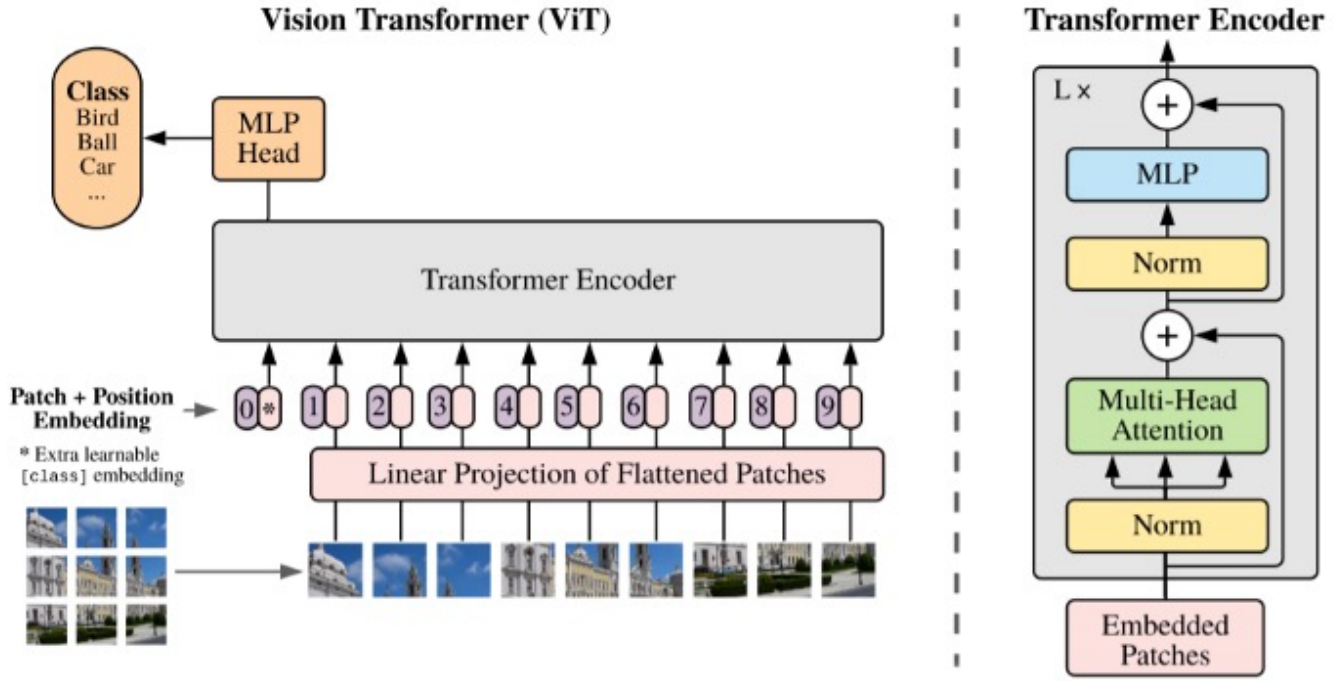


As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".

Line by line code implementation in the colab:
https://colab.research.google.com/drive/1tm0_Usqkavr0h1Jk0f-ukcykI78xmcfW

https://jalammar.github.io/illustrated-transformer/

From NLP to Vision Transformer (ViT)



Supervised pretraining, SOTA fine-tuning performance for downstream classification tasks

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1\mathbf{E}; \mathbf{x}_p^2\mathbf{E}; \cdots; \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \ \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D} \tag{1}$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \ldots L \tag{2}$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \quad \ell = 1 \ldots L \tag{3}$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \tag{4}$$

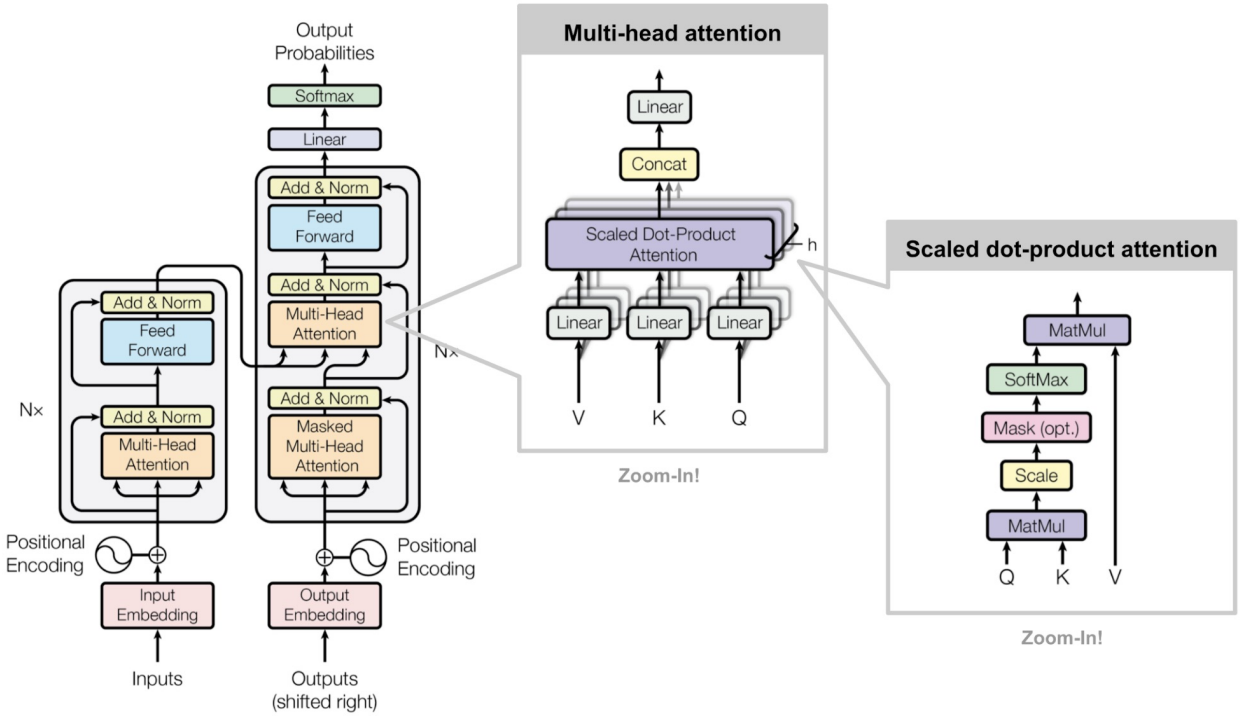
- Image/CNN feature map: $X \in R^{H \times W \times C}$
  - As a sequence of non-overlap patches of $P \times P$ pixels: $X_p \in R^{N \times (P^2 \cdot C)}, N = \frac{HW}{P^2}$
  - A patch: $x_p^i \in R^{P^2 \cdot C}$

- The Transformer uses constant latent vector size D through all of its layers, so we flatten the patches and map to D dimensions with a trainable linear projection (Eq.1).
- Similar to BERT's [class] token, we prepend a learnable embedding to the sequence of embedded patches ($z_0^0 = x_{class}$), whose state at the output of the Transformer encoder (z_L^0) serves as the image representation y (Eq.4)
- As an alternative to raw image patches, the input sequence can be formed from feature maps of a CNN.

Dosovitskiy et al, AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE, https://arxiv.org/pdf/2010.11929.pdf

Code of ViT and Variants: see 3rdparty/pytorch-image-models/timm/models/vision_transformer.py

Transformer



- Pros:
  - *Inductive bias free* – e.g., in contrast to the locality assumption of Convolution
  - *Domain agnostic* – as long as the input is tokenized
  - *Data specificity* – e.g., in contrast to the fixed kernel of convolution in inference.
- Cons:
  - *Permutation invariant*, and thus entailing position encoding
  - *Quadratic complexity* in memory and computation
  - *Lack of inductive bias* – no easy way of encoding domain-specific information.

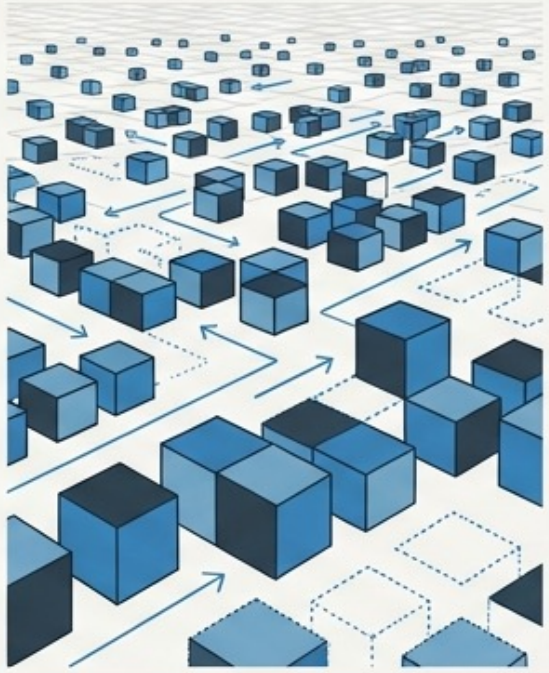
The Ground-Truth Bottleneck

Supervised Learning



- Human supervision is exceptionally expensive (demands manuals, GUIs, and dedicated hiring pipelines).
- Deep Neural Networks (ResNets, DenseNets) are highly over-parameterized and demand massive data.
- Domain expertise constraints severely limit scale in medicine and law.

Unlabeled Data



- Zero explicit labeling cost.
- Infinite volume of images, video, and text available passively.

"*Supervision is the opium of the AI researcher.*" — Jitendra Malik

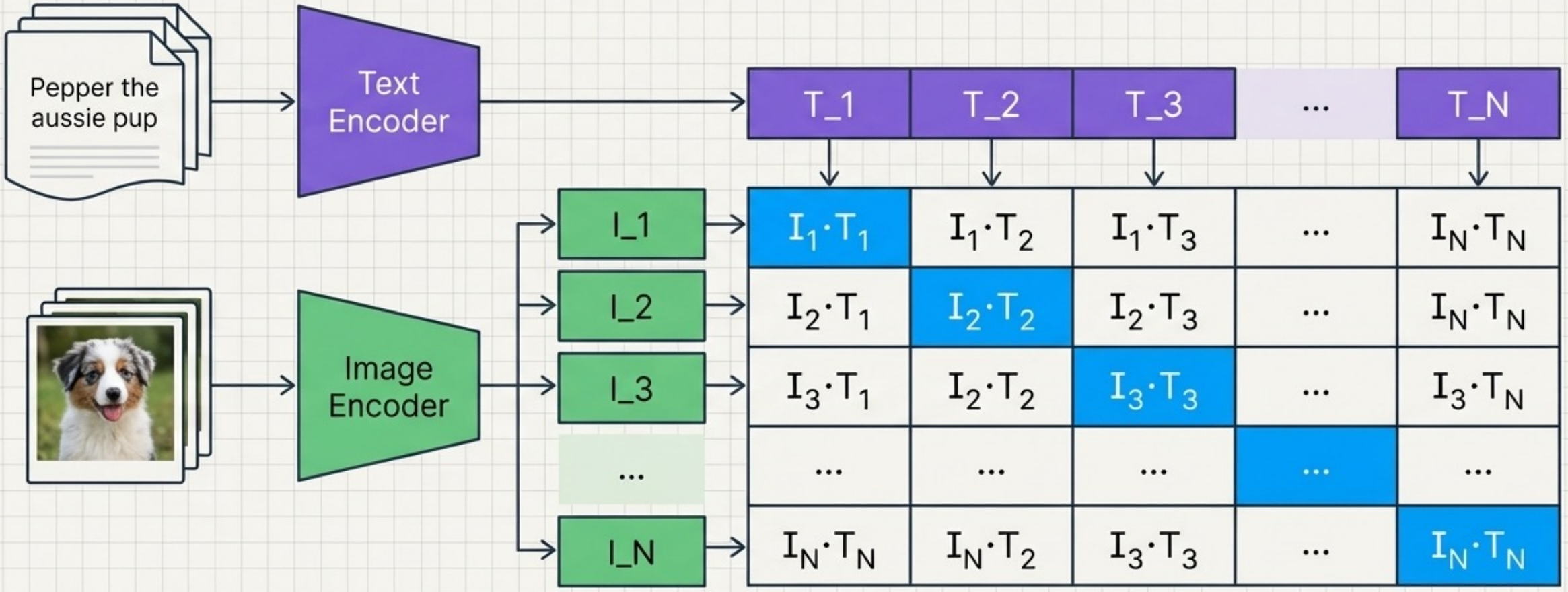
NotebookLM

Transfer Learning: Reuse a Pretrained Network

- **Why:** labels are expensive. A network trained on ImageNet (1.28M photos) already knows edges, textures, parts and objects.
- **Linear probe:** freeze the backbone and train only a new last layer. Cheap; works with a few labels per class.
- **Fine-tuning:** keep training some or all layers with a small learning rate. Better once you have more labels.
- **From scratch:** only with lots of data, or a domain far from natural photos.
- **Notebook 26, CIFAR-10 (probe vs. scratch):** 59% vs. 25% at 5 labels/class; 74% vs. 38% at 100.

Scaling Up: Multimodal Contrastive Pre-training (CLIP)

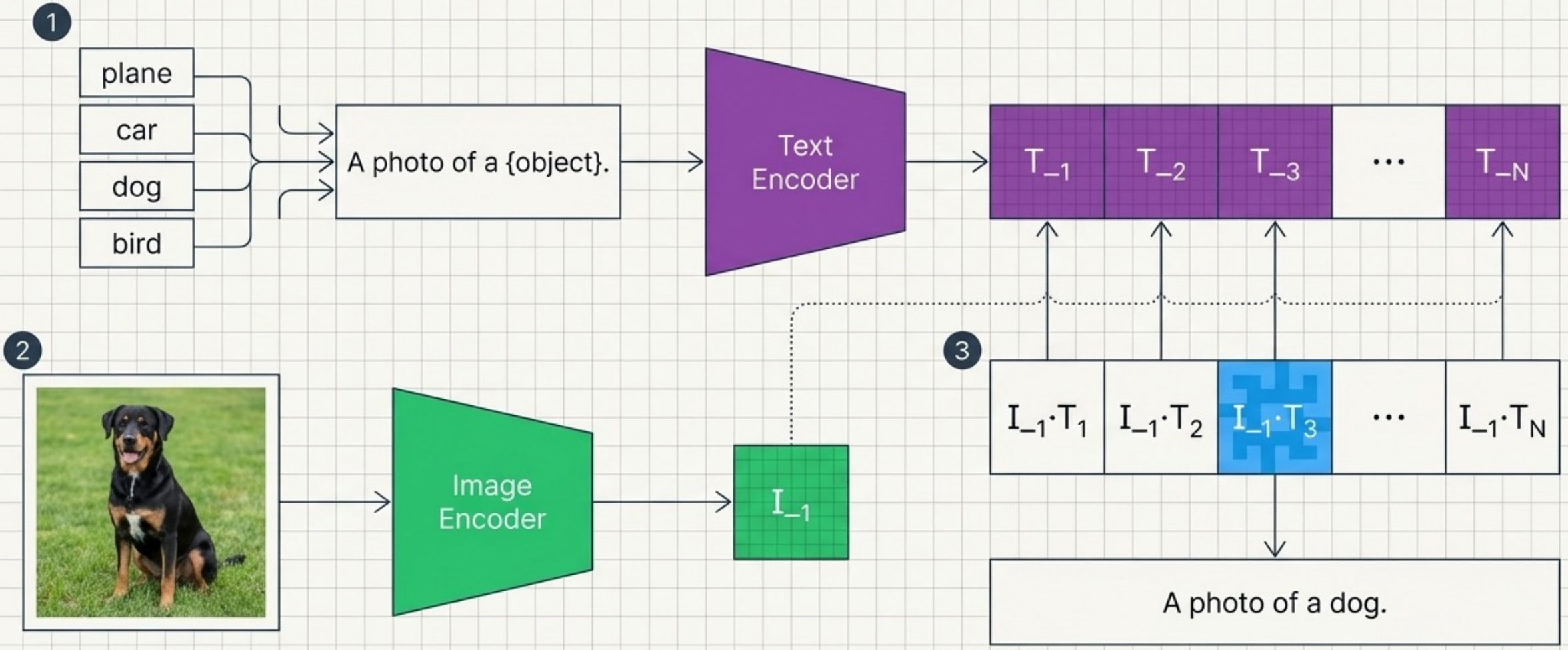
Learning state-of-the-art representations from scratch using 400 million unstructured (image, text) pairs collected from the internet.



NotebookLM

The Zero-Shot Revolution

By combining visual and linguistic representations in a shared space, models can categorize entirely unseen images without traditional fine-tuning.



NotebookLM

A Recipe for Your Course Project

- Start from a pretrained backbone (torchvision, timm, Hugging Face): e.g. ResNet-50, ConvNeXt, ViT, DINOv2.
- Match its preprocessing: input size and the ImageNet mean/std normalization it was trained with.
- Baseline first: a linear probe on frozen features. Then fine-tune with a learning rate about 10x smaller than for training from scratch.
- Use data augmentation (flips, crops, color jitter) and keep a held-out validation set that you never train on.
- Report the pretrained baseline next to your method, not only a from-scratch baseline.

Summary and Next Lecture

- Convolution = local connectivity + weight sharing: translation equivariance with few parameters. Stride-2 subsampling partly breaks it (aliasing).
- Depth needs help: ReLU, normalization, and residual connections (ResNet) make deep ConvNets trainable.
- Self-attention relates every patch to every other patch with content-dependent weights; ViT treats an image as 16x16 patches and learns spatial structure from data.
- Pretrained networks transfer: when labels are scarce, reuse one instead of training from scratch.
- **Next (L23, Mon 11/9):** object classification in practice: benchmarks, augmentation, fine-tuning, evaluation.